



DIGITALBROS
GAME ACADEMY

Corso c++ avanzato

Obiettivi del corso

- Ereditarietà, polimorfismo e interface
- Smart pointers
- Move semantic e RAI
- Lambda
- Eccezioni
- Standard library (2 lezioni)
- Multithreading e parallelismo
- Design pattern

Cristiano Chieppa

- Laureato in scienze dell'informazione
- Sviluppo software dal 1988 (Commodore 64)
- Dal 2015 titolare della Smart Security Software
- Scrivo codice in C++ moderno e Rust
- Ho sviluppato prodotti e servizi in tanti ambiti: banche, medicale, automotive, AI



Ereditarietà

Usiamo il codice scritto da altri

Concetto generale

Cosa è

- L'ereditarietà è un meccanismo per cui una classe (classe derivata) acquisisce interfaccia e implementazione da un'altra classe (classe base).

```
class Shape {  
public:  
    virtual double area() const = 0;  
    virtual ~Shape() = default;  
};  
  
class Circle : public Shape {  
private:  
    double radius_;  
public:  
    Circle(double r) : radius_(r) {}  
    double area() const override {  
        return 3.14159 * radius_ * radius_;  
    }  
};
```

Benefici

- Riutilizzo del codice: Non ripetere implementazione
- Polimorfismo: Stessa interfaccia, comportamento diverso
- Gerarchia semantica: Modella relazioni "è un" nel dominio
- Estensibilità: Aggiungere nuovi tipi senza modificare codice esistente

Quando Usarla:

L'ereditarietà modella una relazione "è un" (is-a):

- Circle **è un** Shape ✓
- Vehicle **è un** Animal ✗
- Dog **è un** Animal ✓

Specificatori di accesso

```
class Base {  
    public:  
        int pub;    // accessibile ovunque  
    protected:  
        int prot;   // accessibile in Base e derivate  
    private:  
        int priv;   // accessibile solo in Base  
};
```

```
// Public inheritance  
class Derived1 : public Base {  
    // pub rimane public  
    // prot rimane protected  
    // priv NON è accessibile  
};  
  
// Protected inheritance  
class Derived2 : protected Base {  
    // pub diventa protected  
    // prot rimane protected  
    // priv NON è accessibile  
};  
  
// Private inheritance (raro!)  
class Derived3 : private Base {  
    // pub diventa private  
    // prot diventa private  
    // priv NON è accessibile  
};
```



Polimorfismo

Lo stesso nome, effetti diversi

Concetto generale

Cosa è

Stessa interfaccia,
comportamento diverso a
seconda dell'oggetto.

```
#include <iostream>

// classe astratta, non può essere istanziata
class Shape {
public:
    [[nodiscard]] virtual double area() const = 0;
    virtual ~Shape() = default;
};
```

```
class Circle final : public Shape {
private:
    double radius_;
public:
    explicit Circle(double r) : radius_(r) {}

    [[nodiscard]] double area() const override {
        return 3.14159 * radius_ * radius_;
    }
};

class Rectangle final : public Shape {
private:
    double width_;
    double height_;
public:
    Rectangle(double w, double h) :
        width_(w), height_(h) {}

    [[nodiscard]] double area() const override {
        return width_ * height_;
    }
};
```

Esempio

```
int main() {  
    Circle c(5.0);  
    Shape* s = &c;  
    std::cout << s->area(); // Output: 78.5 ✓  
    // Runtime dispatch: chiama Circle::area()  
  
    Rectangle r(4.0, 6.0);  
    s = &r;  
    std::cout << s->area(); // Output: 24 ✓  
    // Runtime dispatch: chiama Rectangle::area()  
}
```

Distruttore nella ereditarietà: il problema

```
class Base {
public:
    Base() { std::cout << "Base ctor\n"; }
    ~Base() { std::cout << "Base dtor\n"; } // NO virtual!
};

class Derived : public Base {
public:
    Derived() { std::cout << "Derived ctor\n"; }
    ~Derived() { std::cout << "Derived dtor\n"; }
};

int main() {
    Base* ptr = new Derived();
    // Output: Base ctor
    //      Derived ctor

    delete ptr;
    // Output: Base dtor ✗
    // Manca: Derived dtor
    // MEMORY LEAK: risorse di Derived non liberate!
}
```

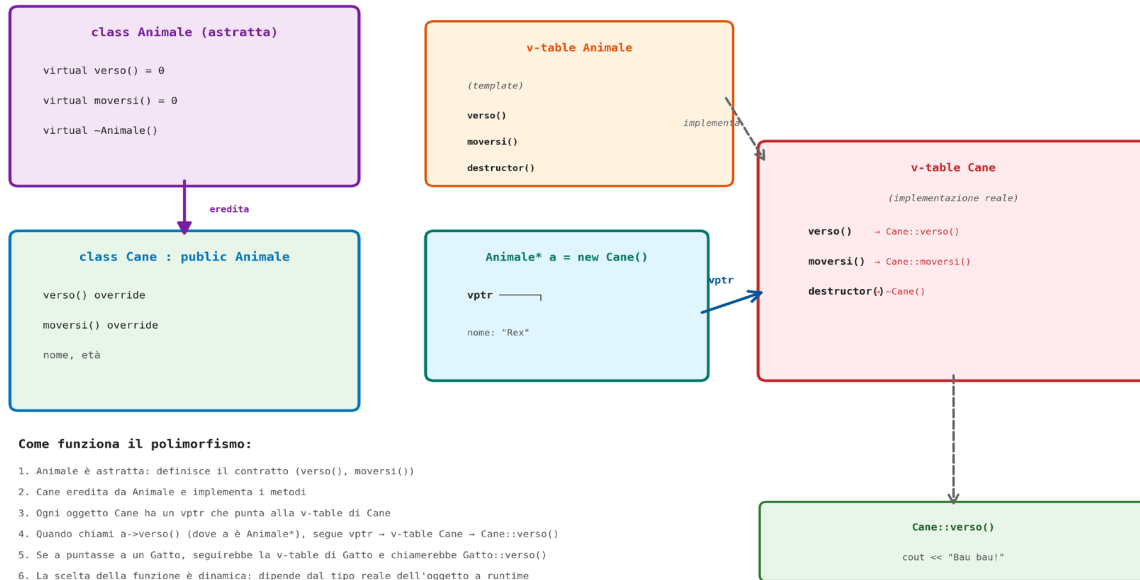
Regola: mettere sempre distruttore virtuale

```
class Base {  
public:  
    virtual ~Base() = default;  
};  
  
class Derived : public Base {  
public:  
    ~Derived() override = default; // optional, ma consigliato  
};  
  
int main() {  
    Base* ptr = new Derived();  
    delete ptr;  
    // Output: Derived dtor (chain calling Base dtor)  
    //      Base dtor  
    // ✓ Tutto liberato correttamente  
}
```

Come c++ fa polimorfismo

La **virtual table (V-Table)** è ciò che fa funzionare il polimorfismo. Si tratta di una tabella creata automaticamente dal compilatore e ne esiste una per ogni classe che possiede una funzione virtuale

Virtual Table (v-table): La Gerarchia Completa



Polimorfismo statico e dinamico

- Per prestazioni potete fare polimorfismo in modo dinamico (no v-table) e a compile time (type safe)
- Perché si fa? Se ho migliaia di chiamate a un metodo virtual, può essere costoso per l'indirizzamento della v-table
- Il trucco è usare un template che accetta come parametro la classe base
- In gergo tecnico si chiama CRTP - Curiously Recurring Template Pattern.

```
// 1. L'Interfaccia Statica (Base)
template <typename T>
class Nemico {
public:
    void attacca() {
        // La Base "comanda", ma esegue l'implementazione della Derivata
        static_cast<T*>(this)->eseguiAttacco();
    }
};

// 2. Implementazioni specifiche
class Orco : public Nemico<Orco> {
public:
    void eseguiAttacco() { std::cout << "Clava! \n"; }
};

class Arciere : public Nemico<Arciere> {
public:
    void eseguiAttacco() { std::cout << "Freccia! \n"; }
};

// 3. Il "Chiamante" (Funzione Template)
template <typename T>
void eseguiTurnoNemico(Nemico<T>& n) {
    n.attacca(); // Qui avviene la magia: il compilatore sa già chi è T
}

int main() {
    Orco o;
    Arciere a;
    eseguiTurnoNemico(o); // Output: Clava!
    eseguiTurnoNemico(a); // Output: Freccia!

    return 0;
}
```

Il tranello dell'object slicing

- Quando si usa ereditarietà e polimorfismo si può cadere nel tranello dell'object slicing: la copia per valore di un oggetto derivato.

```
class Messaggio {
public:
    virtual void mostra() const { std::cout << "Notifica generica\n"; }
    virtual ~Messaggio() = default; // Best practice: sempre distruttore virtuale [cite: 66, 111]
};

class MessaggioPremio : public Messaggio {
public:
    int monete = 100;
    void mostra() const override { std::cout << "HAI VINTO " << monete << " MONETE!\n"; }
};

// ❌ PROBLEMA: Passaggio per valore (Slicing)
void visualizzaNotifica(Messaggio m) {
    m.mostra();
}

int main() {
    MessaggioPremio premio;
    visualizzaNotifica(premio); // Output: "Notifica generica"
    return 0;
}
```

- Perché è successo?** La funzione ha creato una copia m locale. La parte dell'oggetto che conteneva *monete* e il puntatore alla V-Table di MessaggioPremio sono stati eliminati.

Best practice

- Passate sempre per reference o puntatore una classe base per evitare object slicing:

//  SOLUZIONE 1: Riferimento (L'approccio più pulito per funzioni di lettura)

```
void visualizzaNotificaOk(const Messaggio& m) {  
    m.mostra();  
}
```

//  SOLUZIONE 2: Smart Pointer (L'approccio moderno per gestire la proprietà)

```
void visualizzaNotificaSmart(const std::unique_ptr<Messaggio>& m) {  
    m->mostra();  
}
```


Ereditarietà multipla...

```
// Prima classe base
class Veicolo {
public:
    void accelera() {
        cout << "Accelerando..." << endl;
    }
};

// Seconda classe base
class Anfibio {
public:
    void nuota() {
        cout << "Nuotando..." << endl;
    }
};

// Classe derivata che eredita da entrambe
class Idrovolante : public Veicolo, public Anfibio {
public:
    void vola() {
        cout << "Volando!" << endl;
    }
};
```

```
int main() {
    Idrovolante mezzo;

    mezzo.accelera(); // da Veicolo
    mezzo.nuota();    // da Anfibio
    mezzo.vola();     // metodo proprio

    return 0;
}
```

...e i problemi che comporta

```
class Mezzo { // classe base comune
public:
    void accendi() {
        cout << "Motore acceso..." << endl;
    }
};

class Veicolo : public Mezzo {
public:
    void accelera() {
        cout << "Accelerando..." << endl;
    }
};

class Anfibio : public Mezzo {
public:
    void nuota() {
        cout << "Nuotando..." << endl;
    }
};

class Idrovolante : public Veicolo, public Anfibio {
public:
    void vola() {
        cout << "Volando!" << endl;
    }
};

int main() {
    Idrovolante mezzo;
    mezzo.accendi(); // PROBLEMA: quale Mezzo? da Veicolo o da Anfibio?
    return 0;
}
```

```
class Veicolo : virtual public Mezzo { /* ... */ };
class Anfibio : virtual public Mezzo { /* ... */ };
```

Come si risolve

```
class Mezzo {  
public:  
    void accendi() { cout << "Motore acceso..." << endl; }  
};  
  
// Usa 'virtual public' invece di solo 'public'  
class Veicolo : virtual public Mezzo {  
public:  
    void accelera() { cout << "Accelerando..." << endl; }  
};  
  
// Anche qui usa 'virtual public'  
class Anfibio : virtual public Mezzo {  
public:  
    void nuota() { cout << "Nuotando..." << endl; }  
};  
  
// Ora Idrovolante avrà una sola istanza di Mezzo  
class Idrovolante : public Veicolo, public Anfibio {  
public:  
    void vola() { cout << "Volando!" << endl; }  
};
```

Perché va evitata l'ereditarietà multipla

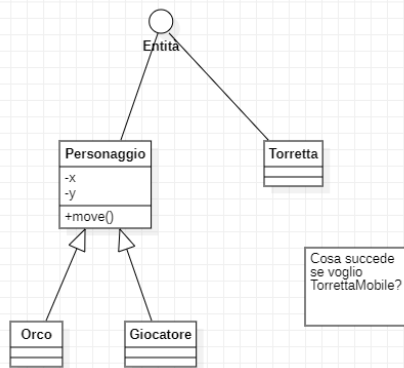
- **Complessità e manutenzione** - Il codice diventa difficile da capire e mantenere. Una classe derivata deve conoscere l'intera gerarchia delle due classi base.
- **Fragilità** - Cambiamenti nelle classi base possono rompere inaspettatamente la derivata. Ad esempio, aggiungere un metodo in una classe base può creare conflitti.
- **Confusione semantica** - Spesso l'ereditarietà multipla riflette cattiva progettazione: una classe che eredita da due genitori senza un concetto comune chiaro probabilmente dovrebbe essere riprogettata.

Alternativa: usare composizione:

```
class Idrovolante {  
    Veicolo veicolo;  
    Anfibia anfibia;  
};
```

Preferite composizione a ereditarietà

- Dove possibile, preferite la composizione di oggetti.

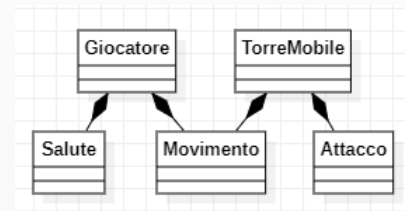


```
// 1. Definiamo i comportamenti
struct Movimento { float velocita; };
struct Salute { int hp; };
struct Attacco { int danno; };

class Torretta {
public:
    std::unique_ptr<Movimento> sistema_arma;
};

class Giocatore {
    std::unique_ptr<Salute> sistema_salute;
    std::unique_ptr<Movimento> sistema_movimento;
};

class TorrettaMobile {
    std::unique_ptr<Movimento> sistema_movimento;
    std::unique_ptr<Attacco> sistema_arma;
};
```



Piccola anticipazione agli smart pointer per esercizi

- Non usate raw pointer (`int *i`) perché dovete sempre gestire `new/delete`
- Per le esercitazioni usate:
 - `Unique_ptr<T>`: esprimono il concetto di ownership, non posso dividerli tra funzioni
 - `Shared_ptr<T>`: esprimono la condivisione con il conteggio dei riferimenti
 - `Make_unique`
 - `Make_shared`

```
#include <iostream>
#include <memory>

void print(std::unique_ptr<int> ptr) {
    std::cout << *ptr << '\n';
}

int main() {
    // Create a unique pointer to an integer
    auto intPtr = std::make_unique<int>(42);

    // Use the unique pointer
    int value = *intPtr;
    // intPtr will be automatically deleted when it goes out of scope

    print(std::move(intPtr)); // Transfer ownership to the print function

    auto strPtr = std::make_shared<std::string>("hello world");
    // Use the shared pointer
    std::string strValue = *strPtr;
    // strPtr will be automatically deleted when the last reference goes out of scope
    return 0;
}
```

Classi astratte

Cosa è

E' un **contratto**. Dicono alle classi derivate "**devi implementare questi metodi**". Non sono istanziabili!

```
class Animale {  
    public:  
        virtual void verso() = 0; // Metodo virtuale  
        puro  
        virtual ~Animale() {}  
};
```

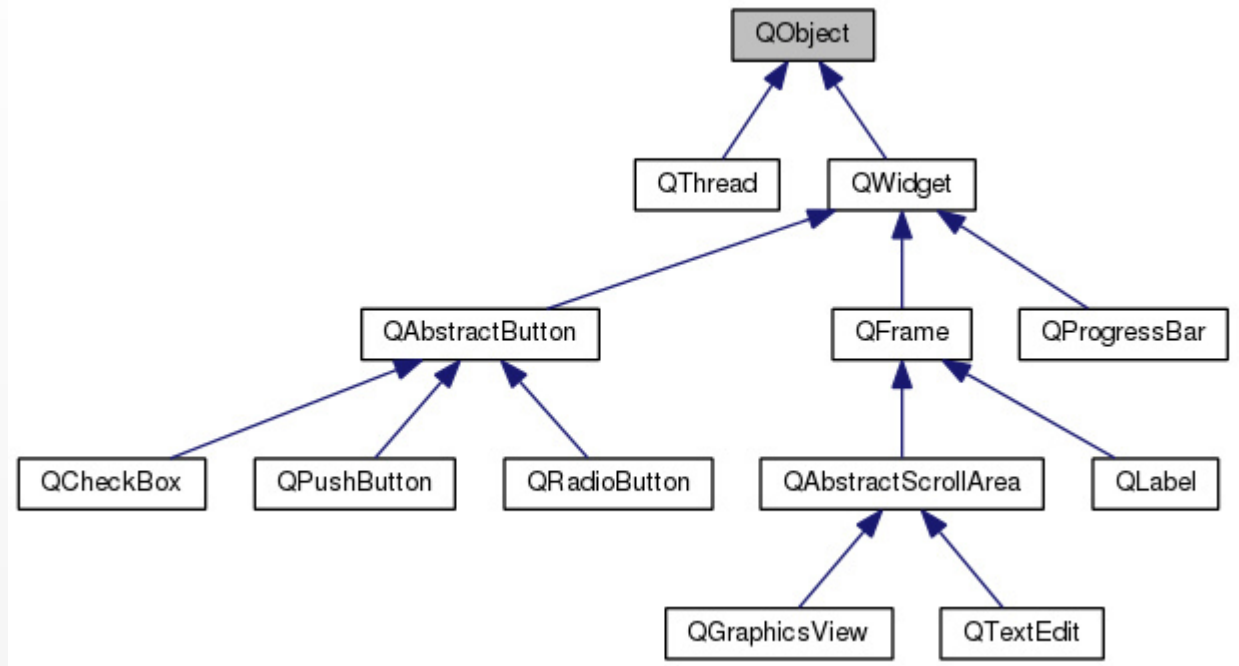
```
class Cane : public Animale {  
    public:  
        void verso() override { std::cout << "Bau!\n"; }  
};  
  
class Gatto : public Animale {  
    public:  
        void verso() override { std::cout << "Miao!\n"; }  
};
```

Classi astratte

Quando si usano

Si usano quando devo creare una gerarchia di classi per una libreria. E' un lavoro che richiede molto design.

A lato, un esempio [Qt](#)



Best practice

- Regola del 5: se possibile dichiarate default o delete i Xctors
- Usate `[[nodiscard]]` e `[[maybe_unused]]` per evitare errori
- Non usate ereditarietà multipla, è il male!
- Nelle classi padre mettete sempre il distruttore virtuale
- Preferite sempre la composizione all'ereditarietà
- Mai puntatori raw! Sempre smart pointer (lo vedremo in seguito)
- Usate sempre *override* per i metodi virtuali ereditati
- Usate sempre metodi *const* ove possibile (sempre nei getters)
- Usate *final* dove si conclude la gerarchia.

Esercizio

- Obiettivo: Creare una gerarchia Nemico -> Orco, Arciere.
- Compito:
 - Definire un metodo virtual void attacca() = 0 nella classe base Nemico.
 - Implementare attacchi diversi nelle classi derivate.
 - Il Test: Creare un `std::vector<std::unique_ptr<Nemico>>` e popolarlo con diversi tipi di nemici.
- Iterare sul vettore chiamando `attacca()` e osservare come il comportamento cambi dinamicamente.
- Bonus: Provare a togliere virtual dal distruttore della base e usare un semplice `cout` per mostrare il memory leak degli oggetti derivati.

Esercizio

- Obiettivo: Risolvere il problema della "Torretta Mobile" senza usare l'ereditarietà multipla.
- Compito:
 - Creare tre struct semplici: Posizione, Salute, Attacco.
 - Creare una classe Entita che contiene std::unique_ptr a questi componenti.
 - Nel main, istanziare tre "Entità":
 - Un Muro: ha solo Salute.
 - Una Torretta: ha Salute e Attacco.
 - Un Drone: ha Posizione, Salute e Attacco.



Move semantic e RAII

Non copiare, spostare e non pulire!

Lvalue e Rvalue

- Lvalue sta per left value. E' una variabile che è «battezzata».
 - Es: `int x = 5;`
- Rvalue sta per right value. E' una variabile che non ha nome.
 - Es: `to_lower(get_string())`
- Ancora, lvalue ha una durata nello scope, rvalue muore subito dopo uso
- Un oggetto definito `const` non è movibile

Costruttore di copia e di assegnamento

```
class DataBuffer {
private:
    std::vector<int> data;

public:
    // Costruttore normale
    DataBuffer(size_t size) {
        data.resize(size);
        std::cout << "✓ Costruttore: allocati " << size << " elementi\n";
    }

    // Copy constructor (COSTOSO)
    DataBuffer(const DataBuffer& other) : data(other.data) {
        std::cout << "⚠ Copy constructor: copiati " << data.size() << " elementi\n";
    }

    // Copy assignment (COSTOSO)
    DataBuffer& operator=(const DataBuffer& other) {
        if (this != &other) {
            data = other.data;
            std::cout << "⚠ Copy assignment: copiati " << data.size() << " elementi\n";
        }
        return *this;
    }

    // Destructor
    ~DataBuffer() {
        std::cout << "X Destructor: liberati " << data.size() << " elementi\n";
    }
};
```

```
int main() {
    std::cout << "=== SENZA MOVE SEMANTICS ===\n\n";

    {
        DataBuffer buf1(5);
        DataBuffer buf2 = buf1; // Copy constructor - COSTOSO!
    }

    return 0;
}
```

Output:

=== SENZA MOVE SEMANTICS ===

Costruttore: allocati 5 elementi

⚠ Copy constructor: copiati 5 elementi

X Destructor: liberati 5 elementi

X Destructor: liberati 5 elementi

Move e assignment

```
class DataBuffer {
private:
    std::vector<int> data;

    ...
    // Move constructor
    DataBuffer(DataBuffer&& other) noexcept :
        data(std::move(other.data)) {
        std::cout << "→ Move constructor: " << data.size()
            << " elementi trasferiti (veloce!)\n";
    }

    // Move assignment
    DataBuffer& operator=(DataBuffer&& other) noexcept
    {
        if (this != &other) {
            data = std::move(other.data);
            std::cout << "→ Move assignment: " << data.size()
                << " elementi trasferiti (veloce!)\n";
        }
        return *this;
    }
};
```

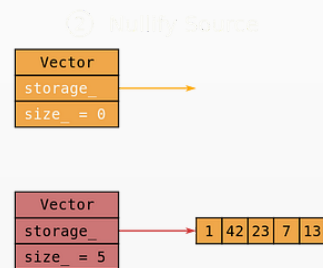
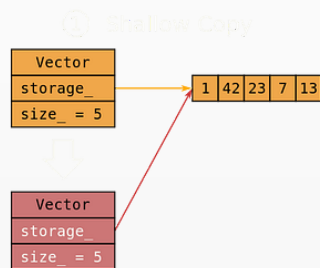
```
int main() {
    std::cout << "\n--- Caso : Move constructor con std::move ---\n";
    {
        DataBuffer buf1(500);
        DataBuffer buf2 = std::move(buf1); // Move constructor
    }

    std::cout << "\n--- Caso : Move assignment ---\n";
    {
        DataBuffer buf1(400);
        DataBuffer buf2(100);
        buf2 = std::move(buf1); // Move assignment
    }

    return 0;
}
```

Cosa succede

```
class Vector {  
private:  
    int* storage_; // Puntatore alla memoria allocata  
    size_t size_;  // Numero di elementi  
  
public:  
    Vector(Vector&& other) noexcept  
        : storage_(other.storage_)  
        , size_(other.size_)  
    {  
        // Nullify  
        other.storage_ = nullptr;  
        other.size_ = 0;  
    }  
    ...  
};
```



Perché usare la semantica del move

- Perché è veloce! Evita di fare copie inutili della memoria. Sposta solo i riferimenti
- Il problema è che una volta mosso un oggetto, non posso più usarlo.
- Molto comune con le lambda
- Standard library usa tanto move semantic

Oggetti mossi

```
int main() {
    std::vector<int> v1 = {1, 2, 3, 4, 5};
    std::cout << "v1: " << v1.size() << " elementi\n";

    // Facciamo il move
    std::vector<int> v2 = std::move(v1);

    std::cout << "Dopo il move:\n";
    std::cout << "v2: " << v2.size() << " elementi\n";
    std::cout << "v1: " << v1.size() << " elementi\n";
    std::cout << "(SVUOTATO!)\n\n";

    // ✓ Questo funziona
    std::cout << "v2[0] = " << v2[0] << "\n";

    // ✗ Questo NON funziona come ti aspetti
    std::cout << "v1[0] = " << v1[0] << "\n"; // UB! v1 è vuoto

    return 0;
}
```

Output:

v1: 5 elementi

Dopo il move:

v2: 5 elementi

v1: 0 elementi (SVUOTATO!)

v2[0] = 1

v1[0] = 0 (forse..., ma è UB)

Funzioni con rvalue e lvalue come parametri

```
class Buffer {  
private:  
    std::vector<int> data;  
  
public:  
    Buffer(size_t size) {  
        data.resize(size);  
        std::cout << "✓ Buffer creato con " << size << " elementi\n";  
    }  
};  
  
// FUNZIONE CHE PRENDE UN RVALUE  
void process(Buffer&& buf) {  
    std::cout << "→ process() riceve un rvalue (temporaneo)\n";  
    std::cout << " Posso muovere le risorse senza copia!\n";  
}  
  
// Per confronto: funzione che prende un lvalue  
void process_copy(const Buffer& buf) {  
    std::cout << "⚠ process_copy() riceve un lvalue (deve copiare)\n";  
}
```

```
int main() {  
    std::cout << "=== FUNZIONE CHE PRENDE RVALUE ===\n\n";  
  
    std::cout << "--- Caso 1: Passiamo un rvalue temporaneo ---\n";  
    process(Buffer(100)); // Buffer(100) è un rvalue temporaneo ✓  
  
    std::cout << "\n--- Caso 2: Passiamo un lvalue con std::move ---\n";  
    Buffer buf(200);  
    process(std::move(buf)); // std::move converte lvalue a rvalue ✓  
  
    std::cout << "\n--- Caso 3: Passiamo un lvalue senza std::move ---\n";  
    Buffer buf2(300);  
    // process(buf2); // X ERRORE! buf2 è lvalue, process() vuole rvalue  
  
    std::cout << "X process(buf2) non compila - buf2 è un lvalue!\n";  
    std::cout << "✓ process_copy(buf2) compila - accetta lvalue\n\n";  
  
    process_copy(buf2); // Questo funziona  
  
    return 0;  
}
```

La buona notizia

- Se in una classe avete tutti dati membri che supportano la semantica del move, potete evitare di scrivere costruttori e operatore di assegnamento della move semantic.

```
class Person {  
private:  
    std::string name;  
    std::vector<int> ages;  
  
public:  
    Person(const std::string& n, size_t size)  
    { name(n) {  
        ages.resize(size);  
        std::cout << "✓ Person creato: " << name << "\n";  
    }  
}
```

- Ma attenzione alla regole del 5! In tal caso, basta definirli come default.

```
// Copy (di solito non serve, ma lo dichiariamo)  
Person(const Person&) = default;  
Person& operator=(const Person&) = default;  
  
// Move (IMPORTANTE: dichiaralo come default!)  
Person(Person&&) = default;  
Person& operator=(Person&&) = default;
```

La regola del 5: o tutti o nessuno

La Rule of Five

Se scrivi **uno qualsiasi** dei seguenti 5 special members:

1. Destructor
2. Copy constructor
3. Copy assignment operator
4. Move constructor
5. Move assignment operator

Allora devi **scrivere (o dichiarare esplicitamente) tutti gli altri 4.**

Perché?

- Quando scrivi uno di questi special members, il compiler assume che stai gestendo qualcosa di "speciale" (come memoria dinamica, risorse, ecc.) e **disabilita l'auto-generazione degli altri.**

RAII

- RAII = **R**esource **A**cquisition **I**s **I**nitialization
- La risorsa vive quanto l'oggetto che lo gestisce: tipico esempio, gli smart pointers

Senza:

```
int main() {  
    FILE* f = fopen("dati.txt", "w");  
    if (!f) return -1;  
  
    se_qualcosa_va_male(); // LEAK se lancia eccezione!  
  
    fclose(f); // Potrebbe non essere mai raggiunto  
}
```

Con:

```
int main() {  
    File f("dati.txt");  
    se_qualcosa_va_male(); // File chiuso automaticamente  
}
```

Best practice

- Se scrivete metodi di move, ricordate di lasciare lo stato nullo se necessario.
- Non accedete mai a indici di collezioni di oggetti mossi (UB!).
- Ricordate sempre la regola del 5.
- Usate RAll quando dovete essere sicuri di liberare risorse.
- Se avanza tempo vediamo la universal reference.



Smart pointers

Non perdiamo la memoria

Il problema

Allocazione

- Nel c++ < 11 per allocare la memoria si usava l'operatore ***new***:
- `int *ptr = new int(42)`

Deallocazione

- Dovevamo sempre ricordarci di usare l'operatore ***delete*** per liberare la memoria:
- `delete ptr`

Leak

- Uno dei più grandi problemi del c++ è il memory leak, allocare senza deallocare.
- Facile farlo!

La soluzione

- Un nuovo oggetto, smart pointer, che cancella da solo la memoria:
 - Unique_ptr
 - Shared_ptr
 - Weak_ptr

Vantaggi degli Smart Pointer

- ✓ No memory leaks - Memoria sempre liberata
- ✓ No double-free - Non puoi sbagliare
- ✓ Exception safe - Se un'eccezione è lanciata, memoria ancora libera
- ✓ Move semantics - Trasferimento di proprietà efficiente
- ✓ RAI - "Resource Acquisition Is Initialization"

Unique_ptr

- Risponde a questa esigenza: qualcuno possiede in esclusiva un oggetto e nessun'altro può prenderlo. Posso solo trasferire la proprietà.

```
struct Risorsa {};  
  
void esempio_base() {  
    std::unique_ptr<int> ptr = std::make_unique<int>(42);  
    // Uso automatico, nessun delete necessario  
    std::cout << *ptr << '\n';  
}  
  
std::unique_ptr<Risorsa> crea_risorsa() {  
    return std::make_unique<Risorsa>();  
}  
  
void consuma_risorsa(std::unique_ptr<Risorsa> r) {  
    // Ora 'r' possiede la risorsa  
}  
  
// Uso:  
int main() {  
    auto res = crea_risorsa();  
    consuma_risorsa(std::move(res)); // res diventa nullptr  
}
```

Shared_ptr

- Risponde a questa esigenza: un oggetto è condiviso tra più elementi. Usa il conteggio dei riferimenti al suo interno. Quando contatore = 0, distrugge oggetto (delete)

```
struct Risorsa {  
    void usa() { std::cout << "In uso\n"; }  
};  
  
int main() {  
    auto ptr1 = std::make_shared<Risorsa>();  
    {  
        auto ptr2 = ptr1; // Copia: contatore = 2  
        auto ptr3 = ptr1; // Copia: contatore = 3  
  
        ptr2->usa();  
        // ptr2 e ptr3 escono dallo scope  
    }  
  
    ptr1->usa(); // Risorsa ancora valida  
  
    // Qui ptr1 esce dallo scope -> distruzione  
}
```

Weak_ptr

- Risponde a questa esigenza: guardo un oggetto, ma non lo possiedo. Non aumenta il conteggio dei riferimenti. Ha un uso particolare: rompere riferimenti circolari.

```
class Parent {  
    std::shared_ptr<Child> figlio;  
};  
  
class Child {  
    std::weak_ptr<Parent> genitore; // Evita ciclo!  
};
```

```
class Risorsa {  
public:  
    void usa() { std::cout << "Usata\n"; }  
};  
  
int main() {  
    std::weak_ptr<Risorsa> weak;  
    {  
        auto shared = std::make_shared<Risorsa>();  
        weak = shared; // weak_ptr non incrementa il contatore  
  
        if (auto ptr = weak.lock()) { // Prova a ottenere shared_ptr  
            ptr->usa(); // OK: risorsa ancora viva  
        }  
    } // shared distrutto -> risorsa distrutta  
  
    if (auto ptr = weak.lock()) {  
        ptr->usa();  
    } else {  
        std::cout << "Risorsa non più disponibile\n";  
    }  
}
```

Custom deleter

- A volte (raramente) non basta il deleter normale, ma serve personalizzarlo per liberare risorse. Possiamo definire un nostro deleter che viene invocato con il distruttore (RAII).
- Necessario con risorse C (librerie solo C ad esempio)

```
// Ipotetica funzione di sistema C
struct TextureGPU { /* ... */ };
void DistruggiTexture(TextureGPU* t); // Funzione che libera memoria video

void caricaLivello() {
    // Creiamo lo smart pointer con il deleter specifico
    std::unique_ptr<TextureGPU, void (*)(TextureGPU*)> texture(
        CaricaTextureSuGPU("player_skin.png"),
        DistruggiTexture
    );

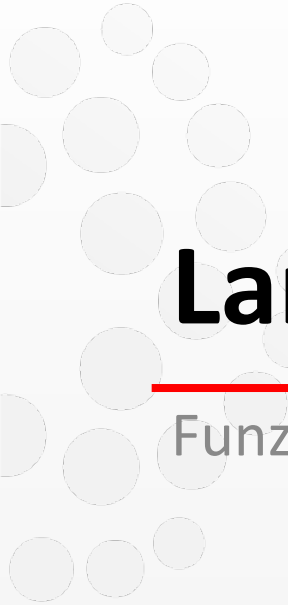
    // Se qualcosa va in errore e usciamo dalla funzione, la texture viene
    // distrutta immediatamente, evitando sprechi di memoria video
}
```

Prima dell'esercitazione

- I metodi noexcept: avvisiamo il compilatore che questo metodo/funzione non darà problemi e quindi ottimizza il codice.
 - `Int getNumero() const noexcept { return x_};`
- `Std::string_view`: è una vista senza copia della string, non modificabile.
 - `int get_len(std::string_view testo) const noexcept { return testo.size();}`

Esercitazione

- Crea una classe Nemico (con nome, punti vita e posizione) e una classe Telecamera:
 - Nemico deve avere un metodo per: ottenere il nome e health point (HP), muoversi nello spazio (x, y) e subire danno (riduce HP)
 - Telecamera deve avere metodo osserva(`std::shared_ptr<Nemico> nemico`) che salva un `std::weak_ptr<Nemico>`.
 - Implementa un metodo `verificaStatoNemico()` che controlla se il nemico è ancora vivo usando `weak_ptr.lock()`:
 - Se vivo: stampa le informazioni del nemico
 - Se morto: stampa "nemico scomparso!"
- Crea una classe Documento (con un titolo e un contenuto testuale) e una classe Editor.
 - Editor deve avere un metodo `creaDocumento(titolo)` che restituisce un `std::unique_ptr<Documento>`.
 - Implementa una funzione `Archivio(std::unique_ptr<Documento> doc)` che sposta il documento in un vettore interno (`std::vector`).



Lambda

Funzioni usa e getta

Cosa sono

- Una lambda è una funzione senza nome (anonima) che puoi scrivere inline nel tuo codice. È una forma compatta e elegante per definire piccole funzioni quando non vale la pena creare una funzione named tradizionale.
- La sintassi: *[cattura](parametri) -> tipo_ritorno { corpo }*
- Sono usate tanto: la standard library ne è piena (quasi sempre parametri)

// Funzione tradizionale

```
int somma(int a, int b) {  
    return a + b;  
}
```

// La stessa cosa, come lambda

```
auto somma_lambda = [](int a, int b) { return a + b; };
```

// Uso

```
int risultato = somma_lambda(3, 5);
```

Le parti di una lambda: la cattura

```
int x = 10;  
int y = 20;
```

```
// [] = non vede nulla dall'esterno  
auto f1 = []() { return 5; };
```

```
// [x] = copia il valore di x al momento della creazione  
auto f2 = [x]() { return x + 5; }; // ritorna 15
```

```
// [&x] = usa il riferimento a x (se x cambia, f3 lo vede)  
auto f3 = [&x]() { return x + 5; };
```

```
// [=] = copia tutti i valori del contesto  
auto f4 = [=]() { return x + y; }; // ritorna 30
```

```
// [&] = usa riferimenti per tutto  
auto f5 = [&]() { return x + y; };
```

```
// Mix: copia x, riferimento per y  
auto f6 = [x, &y]() { return x + y; };
```

La parti di una lambda: parametri

- Parametri

```
auto double = [](int n) { return n * 2; };
```

```
auto somma = [](int a, int b) { return a + b; };
```

```
auto stampa = [](std::string msg) { std::cout << msg << "\n"; };
```

- Ritorno

```
auto f1 = [](int x) { return x * 2; }; // ritorno dedotto (int)
```

```
auto f2 = [](int x) -> double { return x * 2.5; }; // ritorno esplicito (double)
```

Le parti di una lambda: corpo

```
auto printAndReturn = [](int x) {  
    std::cout << "Valore: " << x << "\n";  
    return x * 2;  
};
```

```
printAndReturn(5); // stampa "Valore: 5" e ritorna 10
```

La cattura per valore e riferimento

```
int x = 5;  
  
auto f = [x]() { return x + 1; }; // copia il valore di x  
  
std::cout << f() << "\n"; // stampa 6  
  
x = 100; // cambio x  
  
std::cout << f() << "\n"; // stampa ancora 6, non 101!
```

```
int x = 5;  
  
auto f = [&x]() { return x + 1; }; // riferimento a x  
  
std::cout << f() << "\n"; // stampa 6  
  
x = 100; // cambio x  
  
std::cout << f() << "\n"; // stampa 101
```

La cattura mista e le abbreviazioni

```
int x = 5, y = 10, z = 15;
```

```
// x per valore, y per riferimento, z non catturato  
auto f = [x, &y, z]() { return x + y; }; // z non vedo!
```

```
// abbreviazioni
```

```
int x = 5, y = 10;
```

```
auto f = [=]() { return x + y; }; // copia tutto (x e y)
```

```
auto g = [&]() { return x + y; }; // riferimento per tutto
```

```
auto h =
```

Pattern comuni

- Callback

```
// Una funzione che accetta una callback
void elabora_lista(const std::vector<int>& lista,
                  std::function<void(int)> callback) {
    for (int val : lista) {
        callback(val);
    }
}

int main() {
    std::vector<int> numeri = {1, 2, 3, 4, 5};

    // Passiamo una lambda come callback
    elabora_lista(numeri, [](int n) {
        std::cout << "Numero: " << n << "\n";
    });

    return 0;
}
```


Pattern comuni

- Trasformazioni

```
int main() {  
    std::vector<int> numeri = {1, 2, 3, 4, 5};  
  
    // Raddoppia ogni numero... ma come?  
    // Usiamo std::transform con una lambda  
  
    std::vector<int> raddoppiati(numeri.size());  
  
    std::transform(numeri.begin(), numeri.end(),  
        raddoppiati.begin(),  
        [](int x) { return x * 2; });  
  
    // raddoppiati = {2, 4, 6, 8, 10}  
  
    return 0;  
}
```

Pattern comuni

- Sort

```
int main() {  
    std::vector<std::string> parole = {"ciao", "mondo", "c", "lambdas"};  
  
    // Ordina per lunghezza della stringa (crescente)  
    std::sort(parole.begin(), parole.end(),  
        [](const std::string& a, const std::string& b) {  
            return a.length() < b.length();  
        });  
  
    // parole = {"c", "ciao", "mondo", "lambdas"}  
  
    return 0;  
}
```

Quando usare una lambda

Usale quando:

- La funzione è piccola e semplice
- La usi solo in una posizione
- La usi come callback o con algoritmi STL
- Vuoi evitare di clutter il namespace globale

Non usarle quando:

- La funzione è complessa e lunga (usa una funzione named)
- La usi in molti posti (usa una funzione named)
- Hai bisogno di ricorsione (complesso con lambda)

Esercizi

- Crea una variabile `moltiplicatore = 5`
 - Scrivi una lambda che moltiplica un numero per `moltiplicatore`
 - La lambda deve catturare `moltiplicatore` per valore
 - Usa questa lambda con `std::vector<int>` e stampa i risultati
- Crea un contatore `= 0`
 - Scrivi una lambda che incrementa il contatore catturato per riferimento
 - Chiama la lambda 10 volte e stampa il contatore finale
 - Cosa cambia rispetto a catturare per valore?
- Opzionale: dato un `std::vector<int>` con valori `[1, 2, 3, 4, 5]` usa `std::transform` con una lambda per elevare al quadrato e stampa il risultato



Eccezioni

Quando il panico ci assale

Cosa sono

- Un'**eccezione** è un meccanismo per segnalare che qualcosa è andato storto durante l'esecuzione del programma. Invece di ritornare un codice di errore, interrompi l'esecuzione normale e "lanci" un'eccezione che può essere "catturata" e gestita.

Esempio

// Senza eccezioni: devi verificare codici di errore

```
int dividere(int a, int b, int& risultato) {  
    if (b == 0) {  
        return -1; // Codice di errore  
    }  
    risultato = a / b;  
    return 0; // OK  
}
```

```
int main() {  
    int res;  
    if (dividere(10, 0, res) == -1) {  
        std::cout << "Errore: divisione per zero\n";  
    }  
}
```

// Con eccezioni: il flusso si interrompe e tu gestisci l'errore

```
int dividere(int a, int b) {  
    if (b == 0) {  
        throw std::invalid_argument("Divisione per zero!");  
    }  
    return a / b;  
}
```

```
int main() {  
    try {  
        int res = dividere(10, 0);  
        std::cout << res << "\n";  
    } catch (std::invalid_argument& e) {  
        std::cout << "Errore catturato: " << e.what() << "\n";  
    }  
}
```

Sintassi base

- throw - lancio una eccezione
- try - blocco di tentativo
- catch - cattura dell'eccezione
- Quando un'eccezione viene lanciata, il catch la **cattura** se il tipo corrisponde.
- Se un'eccezione non viene gestita, il programma termina (terminate()).

Esempio completo

```
#include <iostream>
#include <stdexcept>

int dividere(int a, int b) {
    if (b == 0) {
        throw std::invalid_argument("Non puoi dividere per zero!");
    }
    return a / b;
}

int main() {
    try {
        std::cout << "Risultato: " << dividere(10, 2) << "\n"; // OK: 5
        std::cout << "Risultato: " << dividere(10, 0) << "\n"; // Lancia eccezione!
        std::cout << "Questa riga non si esegue\n";
    }
    catch (std::invalid_argument& e) {
        std::cout << "Errore catturato: " << e.what() << "\n";
    }

    std::cout << "Il programma continua\n"; // Questo si esegue
    return 0;
}
```

Tipi di eccezioni standard

- C++ fornisce una gerarchia di eccezioni standard in `<stdexcept>:std::exception` (la base)
 - `std::logic_error`
 1. `std::invalid_argument`
 2. `std::out_of_range`
 3. `std::length_error`
 - `std::runtime_error`
 1. `std::range_error`
 2. `std::overflow_error`
 - `std::bad_alloc` (fuori memoria)

Il metodo what()

- Tutte le eccezioni standard hanno un metodo `what()` che ritorna il messaggio di errore.

```
try{  
    throw std::invalid_argument("Qualcosa non va");  
}  
catch (std::exception& e) { // Usa la classe base!  
    std::cout << "Messaggio: " << e.what() << "\n";  
}
```

Cattura multipla

```
void processa_comando(const std::string& cmd) {  
    if (cmd.empty()) {  
        throw std::invalid_argument("Comando vuoto");  
    }  
    if (cmd == "quit") {  
        throw std::runtime_error("Terminazione richiesta");  
    }  
}  
  
int main() {  
    try {  
        processa_comando("");  
    }  
    catch (std::invalid_argument& e) {  
        std::cout << "Argomento non valido: " << e.what() << "\n";  
    }  
    catch (std::runtime_error& e) {  
        std::cout << "Errore runtime: " << e.what() << "\n";  
    }  
  
    return 0;  
}
```

Ordine dei catch è importante

Metti i tipi più specifici prima e poi i generici

```
try {  
    // ...  
}  
catch (std::invalid_argument& e) { // Specifico  
    // ...  
}  
catch (std::exception& e) { // Generico (base)  
    // ...  
}  
catch (...) { // Qualunque cosa  
    std::cout << "Eccezione sconosciuta!\n";  
}
```

Cattura tutti!

- Non sai quali eccezioni il codice lancerà. `catch (...)` le cattura tutte, ma non puoi accedere al messaggio.

```
try{  
    throw "Una stringa"; // Non è std::exception!  
}  
catch (std::exception& e) {  
    // Non la cattura!  
}  
catch (...) {  
    std::cout << "Ho catturato qualcosa, ma non so cosa!\n";  
}
```

Eccezioni custom

- Puoi creare le tue eccezioni ereditando da `std::exception`:

```
class ErroreDivisione : public std::exception {
public:
    const char* what() const noexcept override {
        return "Errore: divisione per zero!";
    }
};

int dividere(int a, int b) {
    if (b == 0) {
        throw ErroreDivisione();
    }
    return a / b;
}

int main() {
    try {
        dividere(10, 0);
    }
    catch (ErroreDivisione& e) {
        std::cout << e.what() << "\n";
    }
}
```

Garanzia noexcept

- La funzione **non** lancia mai un'eccezione e il compilatore fa ottimizzazioni (no stack per unwind). Ma se per caso lancia... terminate().

```
void stampa_messaggio(const std::string& msg) noexcept {  
    std::cout << msg << "\n";  
}
```


Best practice

- **Usale quando:**

- Ci sono errori che il chiamante **potrebbe** gestire
- Vuoi separare il codice di errore dal codice normale
- Il valore di ritorno ha un significato (non puoi usarlo per segnalare errore)
- Attraversi molti livelli di call stack (stack unwinding)
- Cattura sempre per riferimento: *catch(const std::exception& e)*

- **Non usarle quando:**

- È un flusso di controllo normale (non è un errore)
- L'errore è prevedibile e facile da gestire (per es., if (x < 0))
- Sei in codice di basso livello critico per performance
- Sei in un distruttore



Standard library

Spesso la risposta è nella libreria

Le aree di interesse

- Collection
- Iteratori
- Algoritmi
- Utilità
- Date e time
- Numeric
- Concurrency (multithreading)

Collezioni

- Sequenziali
 - `Std::vector`, `std::array`, `std::deque`, `std::list`
- Associativi
 - `Std::map`, `std::set`
- Non ordinati
 - `Std::unordered_map`, `std::unordered_set`

Accesso random: std::array

- Che cos'è?

- Un array di dimensione fissa. Dimensione nota a compile time.

```
#include <array>
```

```
std::array<int, 3> numeri = {10, 20, 30};
```

```
numeri[0];    // 10 (accesso veloce)
```

```
numeri.at(0); // 10 (con controllo bounds)
```

```
numeri.front(); // 10 (primo)
```

```
numeri.back();  // 30 (ultimo)
```

```
for (const auto &x : numeri) {
```

```
    std::cout << x << " ";
```

```
}
```

Accesso random: std::vector

Che cos'è?

- Un array **dinamico**. Cresce e si rimpicciolisce durante l'esecuzione. Vive nell'heap.

```
#include <vector>
```

```
std::vector<int> vec = {10, 20, 30};
```

```
vec[0]; // 10 (veloce, no controllo)
```

```
vec.at(0); // 10 (con controllo bounds, più sicuro)
```

```
vec.front(); // 10 (primo)
```

```
vec.back(); // 30 (ultimo)
```

```
vec.push_back(4); // Aggiungi alla fine: {10, 20, 30, 4}
```

```
vec.pop_back(); // Togli l'ultimo: {10, 20, 30}
```

```
vec.insert(vec.begin() + 1, 99); // Inserisci in posizione: {10, 99, 20, 30}
```

```
vec.erase(vec.begin() + 1); // Togli in posizione: {10, 20, 30}
```

```
for (const auto& x : vec) {  
    std::cout << x << " ";  
}
```

Accesso random: std::deque

- Che cos'è?
 - Una **coda a doppia estremità** (Double-Ended Queue). Veloce sia per aggiungere/togliere **all'inizio che alla fine**. A differenza di vector, che è veloce solo alla fine.
- Usa vector (95% dei casi)
 - Non sai la dimensione in anticipo
 - Aggiungi/togli principalmente **dalla fine** (push_back, pop_back)
- Usa deque (quando ne hai davvero bisogno)
 - Aggiungi/togli **frequentemente da entrambi i lati**
 - Implementi una **coda reale** (FIFO)

Accesso random: std::deque

```
#include <iostream>
#include <deque>

int main() {
    // Creiamo una deque di interi
    std::deque<int> coda = {2, 3, 4};

    std::cout << "Iniziale: ";
    for (const auto& x : coda) std::cout << x << " ";
    std::cout << "\n\n";

    // Aggiungiamo all'inizio e alla fine
    coda.push_back(5); // Aggiungi alla fine
    coda.push_front(1); // Aggiungi all'inizio

    std::cout << "Dopo push_back(5) e push_front(1): ";
    for (const auto& x : coda) std::cout << x << " ";
    std::cout << "\n";
    std::cout << "Size: " << coda.size() << "\n\n";

    // Accediamo agli elementi
    std::cout << "Front (primo): " << coda.front() << "\n";
    std::cout << "Back (ultimo): " << coda.back() << "\n";
    std::cout << "Elemento in posizione [2]: " << coda[2] << "\n\n";

    // Rimuoviamo dai lati
    coda.pop_front(); // Togli il primo
    coda.pop_back(); // Togli l'ultimo

    std::cout << "Dopo pop_front() e pop_back(): ";
    for (const auto& x : coda) std::cout << x << " ";
    std::cout << "\n";
    std::cout << "Size: " << coda.size() << "\n";

    return 0;
}
```


Accesso sequenziale: std::list

- Una lista concatenata (linked list). Ogni elemento è un **nodo** che contiene:
 - Il valore
 - Un puntatore al nodo successivo
- [1] -> [2] -> [3] -> [4] -> [5] -> nullptr
- Ogni nodo sa dove trovare il prossimo
- Insert/erase in mezzo è **$O(1)$** (basta scollegare e ricollegare i puntatori)
- No accesso random - per arrivare all'elemento 10, devi attraversare i 9 precedenti

Accesso sequenziale: std::list

```
#include <iostream>
#include <list>
int main() {
    // Creiamo una lista di interi
    std::list<int> lista = {1, 2, 3, 4, 5};

    std::cout << "Iniziale: ";
    for(const auto& x : lista) std::cout << x << " ";
    std::cout << "\n\n";

    // Aggiungiamo all'inizio e alla fine
    lista.push_back(6); // Aggiungi alla fine
    lista.push_front(0); // Aggiungi all'inizio

    std::cout << "Dopo push_back(6) e push_front(0): ";
    for(const auto& x : lista) std::cout << x << " ";
    std::cout << "\n";
    std::cout << "Size: " << lista.size() << "\n\n";

    // Accediamo ai lati
    std::cout << "Front (primo): " << lista.front() << "\n";
    std::cout << "Back (ultimo): " << lista.back() << "\n";
    std::cout << "Nota: NO lista[2]! (niente accesso random)\n\n";

    // Troviamo un elemento e lo rimuoviamo
    auto it = lista.begin();
    std::advance(it, 3); // Vai al 4° elemento
    std::cout << "Elemento in posizione 3: " << *it << "\n";
    lista.erase(it); // Rimuovilo (veloce!)

    std::cout << "Dopo erase in posizione 3: ";
    for(int x : lista) std::cout << x << " ";
    std::cout << "\n";
    std::cout << "Size: " << lista.size() << "\n\n";

    // Inseriamo in mezzo (veloce in list!)
    auto it2 = lista.begin();
    std::advance(it2, 2);
    lista.insert(it2, 99); // Inserisci 99 alla posizione 2

    std::cout << "Dopo insert(99) in posizione 2: ";
    for(int x : lista) std::cout << x << " ";
    std::cout << "\n";

    return 0;
}
```

Accociativi: std::map

- Cos'è?

- Una collezione di coppie **chiave**→**valore**, **ordinata**.
- Accedi ai dati **per chiave**, non per posizione.

```
std::map<std::string, int> eta;  
eta["Alice"] = 25;  
eta["Bob"] = 30;
```

```
// Accedi per CHIAVE
```

```
eta["Alice"]; // ritorna 25
```

```
// Non per posizione: non c'è eta[0]
```

Associativi: std::map

```
#include <iostream>
#include <map>

int main() {
    std::map<std::string, int> eta;

    eta["Alice"] = 25;
    eta["Bob"] = 30;
    eta["Charlie"] = 22;

    std::cout << "Alice ha " << eta["Alice"] << " anni\n";
    std::cout << "Bob ha " << eta["Bob"] << " anni\n\n";

    for (const auto& [nome, anni] : eta) {
        std::cout << nome << " -> " << anni << "\n";
    }

    return 0;
}
```

Associativi: std::set

- Una **collezione di valori unici, ordinati**.
 - Diversamente da map (chiave→valore), set contiene solo **valori** (no coppie).

```
std::set<int> numeri = {3, 1, 4, 1, 5};  
// Automaticamente: {1, 3, 4, 5}  
// Duplicati rimossi, ordinati
```

Associativi: std::set

```
#include <iostream>
#include <set>
int main() {
    std::set<int> numeri;

    numeri.insert(3);
    numeri.insert(1);
    numeri.insert(4);
    numeri.insert(1); // Duplicato, non inserito
    numeri.insert(5);

    std::cout << "Numeri (ordinati, unici):\n";
    for (const auto& x : numeri) {
        std::cout << x << " ";
    }
    std::cout << "\n\n";

    // Cerchiamo
    if (numeri.find(3) != numeri.end()) {
        std::cout << "3 trovato!\n";
    }

    if (numeri.find(2) == numeri.end()) {
        std::cout << "2 non trovato!\n";
    }
    std::cout << "\n";

    // Rimuoviamo
    numeri.erase(4);

    std::cout << "Dopo erase(4):\n";
    for (const auto& x : numeri) {
        std::cout << x << " ";
    }
    std::cout << "\nSize: " << numeri.size() << "\n";
    return 0;
}
```

Associativi:: std::unordered_map

- Cos'è?

- Una **mappa chiave→valore** come std::map, ma **senza ordine**.
- Usa una **hash table** internamente: più veloce per accesso.

```
std::unordered_map<std::string, int> eta;  
eta["Alice"] = 25;  
eta["Bob"] = 30;
```

```
// Accesso veloce (O(1) in media)  
eta["Alice"]; // ritorna 25 subito
```

Associativi: come sono implementati

- **std::map:**

- Usa un **albero binario** internamente
- Per trovare una chiave, deve attraversare l'albero
- Tempo: **$O(\log n)$** (logaritmico, costante)

- **std::unordered_map:**

- Usa una **hash table** internamente
- Accesso diretto via hash
- Tempo: **$O(1)$** (istantaneo)

- Per pochi elementi, sempre meglio std::map, usa meno memoria.

Associativi: `std::unordered_set`

• Cos'è?

- Un **set** senza ordine.
- Come `std::set` (valori unici), ma implementato con **hash table** invece di albero. Più veloce, ma senza ordine.

```
std::unordered_set<int> numeri = {3, 1, 4, 1, 5};  
// {1, 3, 4, 5} ma ordine casuale, non ordinato
```

Associativi: std::unordered_set

```
#include <iostream>
#include <unordered_set>
int main() {
    std::unordered_set<int> numeri;

    numeri.insert(3);
    numeri.insert(1);
    numeri.insert(4);
    numeri.insert(1); // Duplicato, non inserito
    numeri.insert(5);

    std::cout << "Numeri (unici, ordine casuale):\n";
    for (const auto& x : numeri) {
        std::cout << x << " ";
    }
    std::cout << "\n\n";

    // Cerchiamo
    if (numeri.find(3) != numeri.end()) {
        std::cout << "3 trovato!\n";
    }

    if (numeri.find(2) == numeri.end()) {
        std::cout << "2 non trovato!\n";
    }
    std::cout << "\n";

    // Rimuoviamo
    numeri.erase(4);

    std::cout << "Dopo erase(4):\n";
    for (const auto& x : numeri) {
        std::cout << x << " ";
    }
    std::cout << "\nSize: " << numeri.size() << "\n";

    return 0;
}
```

Iteratori

- Forward iterator
- Bidirectional iterator
- Random access iterator

Cosa sono gli iteratori

- Un **puntatore generalizzato** ai container. Ti consente di navigare e manipolare gli elementi.
- Pensa a un iteratore come una **freccia che punta** a un elemento:

○ vector: [1] [2] [3] [4] [5]



iteratore punta qui

Operazioni di base

```
std::vector<int> vec = {1, 2, 3, 4, 5};
```

```
auto it = vec.begin(); // Punta al primo elemento  
std::cout << *it;      // Dereferenzia: 1
```

```
++it;                  // Avanza al prossimo  
std::cout << *it;      // Dereferenzia: 2
```

```
--it;                  // Torna indietro  
std::cout << *it;      // Dereferenzia: 1
```

```
if (it != vec.end()) { // end() non è un elemento, è il "dopo l'ultimo"  
    std::cout << "Iteratore valido\n";  
}
```

Iterare con gli iteratori

```
std::vector<int> vec = {1, 2, 3, 4, 5};  
  
for (auto it = vec.begin(); it != vec.end(); ++it) {  
    std::cout << *it << " "; // 1 2 3 4 5  
}
```

- Che è la stessa cosa, più idiomatica di:

```
for (const auto& x : vec) {  
    std::cout << x << " "; // 1 2 3 4 5  
}
```

Quando li usiamo

```
std::vector<int> vec = {1, 2, 3, 4, 5};
```

```
// Trovare un elemento
```

```
auto it = std::find(vec.begin(), vec.end(), 3);
```

```
if (it != vec.end()) {
```

```
    std::cout << "Trovato: " << *it << "\n"; // 3
```

```
}
```

```
// Inserire/eliminare
```

```
vec.insert(vec.begin() + 2, 99); // Inserisci alla posizione 2
```

```
vec.erase(vec.begin() + 2); // Togli dalla posizione 2
```

Tipi di iteratori: random access

```
std::vector<int> vec = {1, 2, 3, 4, 5};
```

```
auto it = vec.begin();
```

```
*it;           // Accedi all'elemento (prende 1)
```

```
++it;          // Avanza di 1 (punta a 2)
```

```
it += 2;       // Avanza di 2 passi (punta a 4)
```

```
it[0];         // Accesso diretto come array, prende 1
```

```
it < vec.end(); // Confronto
```

- **Container:** vector, array, deque

Tipi di iteratori: bidirezionali

```
std::list<int> lista = {1, 2, 3, 4, 5};
```

```
auto it = lista.begin();
```

```
*it;           // Accedi all'elemento
```

```
++it;          // Avanza
```

```
--it;          // Torna indietro
```

```
it += 2;        // ❌ NO! Non puoi saltare
```

```
it[0];          // ❌ NO! Non puoi accedere con [i]
```

- **Container:** list, map, set

Tipi di iteratori: forward (solo avanti)

```
std::forward_list<int> flista = {1, 2, 3};
```

```
auto it = flista.begin();
```

```
*it;           // Accedi
```

```
++it;          // Avanza
```

```
--it;          // ❌ NO! Non puoi tornare indietro
```

```
it += 2;        // ❌ NO! Non puoi saltare
```

- **Container:** forward_list

Esercizio

Contare Frequenza Parole

Hai una lista di parole. Devi contare quante volte **ogni parola appare** e stampare il risultato in ordine alfabetico.

- 1. Usa una map per associare parola → frequenza .
- 2. Itera su tutte le parole
- 3. Per ogni parola, incrementa il contatore nella map
- 4. Stampa la map (che sarà automaticamente ordinata)

```
#include <iostream>
```

```
#include <map>
```

```
#include <string>
```

```
int main() {
```

```
    std::map<std::string, int> frequenza;
```

```
    std::string parole[] = {"ciao", "mondo", "ciao", "test", "mondo", "ciao"};
```

```
    // SCRIVI QUI: itera e incrementa
```

```
    // SCRIVI QUI: stampa il risultato
```

```
    return 0;
```

```
}
```

Esercizio (opzionale)

- Devi creare una **rubrica telefonica** dove puoi associare nomi a numeri di telefono. Poi devi:
 - Cercare il numero di una persona
 - Verificare se una persona è in rubrica

```
#include <iostream>
#include <map>
#include <string>
```

```
int main() {
    std::map<std::string, std::string> rubrica;

    // SCRIVI QUI: aggiungi 3 persone

    // SCRIVI QUI: stampa numero di Alice

    // SCRIVI QUI: verifica se Diana è in rubrica

    return 0;
}
```

Algoritmi

- Ordinamento
- Numerici
- Ricerca
- Modifica

Cosa sono

- Un **insieme di funzioni**, tipicamente per manipolare i dati nei container.
- Lavorano con **iteratori**, quindi funzionano su qualsiasi container (vector, list, map, etc.).
- Usano spesso le lambda passate come parametri

Ricerca: std::find

```
include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> numeri = {10, 20, 30, 40, 50};

    int ricerca = 30;
    auto it = std::find(numeri.begin(), numeri.end(), ricerca);

    if (it != numeri.end()) {
        std::cout << "Trovato: " << *it << "\n";
        std::cout << "Posizione: " << (it - numeri.begin()) << "\n";
    } else {
        std::cout << "Non trovato\n";
    }

    return 0;
}

// Trovato: 30 Posizione: 2
```

Ricerca: std::find_if

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> numeri = {5, 12, 8, 20, 3};

    // Trova il primo numero > 10
    auto it = std::find_if(numeri.begin(), numeri.end(),
        [](int x) { return x > 10; });

    if (it != numeri.end()) {
        std::cout << "Primo numero > 10: " << *it << "\n";
    } else {
        std::cout << "Non trovato\n";
    }

    return 0;
}

// Primo numero > 10: 12
```


Contare le occorrenze: std::count_if

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> numeri = {5, 12, 8, 20, 3, 15};

    // Conta numeri > 10
    int conteggio = std::count_if(numeri.begin(), numeri.end(),
        [](int x) { return x > 10; });

    std::cout << "Numeri > 10: " << conteggio << "\n";

    return 0;
}

// Numeri > 10: 3
```

Ricerca in un container ordinato: std::binary_search

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> numeri = {1, 3, 5, 7, 9, 11, 13};

    int ricerca = 7;

    bool trovato = std::binary_search(numeri.begin(), numeri.end(), ricerca);

    if (trovato) {
        std::cout << ricerca << " trovato\n";
    } else {
        std::cout << ricerca << " non trovato\n";
    }

    return 0;
}

// 7 trovato
```

Ordinamento: std::sort

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> numeri = {5, 2, 8, 1, 9, 3};

    std::cout << "Prima: ";
    for (int x : numeri) std::cout << x << " ";
    std::cout << "\n";

    // Ordina decrescente
    std::sort(numeri.begin(), numeri.end(),
        [](int a, int b) { return a > b; });

    std::cout << "Dopo sort decrescente: ";
    for (int x : numeri) std::cout << x << " ";
    std::cout << "\n";

    return 0;
}

// Prima: 5 2 8 1 9 3 Dopo sort: 1 2 3 5 8 9
```

Ordinamento: std::sort

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

int main() {
    std::vector<std::string> nomi = {"Charlie", "Alice", "Bob", "Diana"};

    std::cout << "Prima: ";
    for (auto& nome : nomi) std::cout << nome << " ";
    std::cout << "\n";

    std::sort(nomi.begin(), nomi.end());

    std::cout << "Dopo sort: ";
    for (auto& nome : nomi) std::cout << nome << " ";
    std::cout << "\n";

    return 0;
}

// Prima: Charlie Alice Bob Dian; Dopo sort: Alice Bob Charlie Diana
```

Invertire: std::reverse

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> numeri = {1, 2, 3, 4, 5};

    std::cout << "Prima: ";
    for (int x : numeri) std::cout << x << " ";
    std::cout << "\n";

    std::reverse(numeri.begin(), numeri.end());

    std::cout << "Dopo reverse: ";
    for (int x : numeri) std::cout << x << " ";
    std::cout << "\n";

    return 0;
}

// Prima: 1 2 3 4 5 Dopo reverse: 5 4 3 2 1
```

Copiare: std::copy

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> origine = {1, 2, 3, 4, 5};
    std::vector<int> copia(5); // Spazio già allocato

    std::copy(origine.begin(), origine.end(), copia.begin());

    std::cout << "Origine: ";
    for (int x : origine) std::cout << x << " ";
    std::cout << "\n";

    std::cout << "Copia: ";
    for (int x : copia) std::cout << x << " ";
    std::cout << "\n";

    return 0;
}

// Origine: 1 2 3 4 5 Copia: 1 2 3 4 5
```

Copiare: std::copy_if

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> numeri = {1, 5, 2, 8, 3, 9, 4, 6};
    std::vector<int> pari;

    // Copia solo i numeri pari
    std::copy_if(numeri.begin(), numeri.end(),
                std::back_inserter(pari),
                [](int x) { return x % 2 == 0; });

    std::cout << "Originali: ";
    for (int x : numeri) std::cout << x << " ";
    std::cout << "\n";

    std::cout << "Pari (con copy_if): ";
    for (int x : pari) std::cout << x << " ";
    std::cout << "\n";

    return 0;
}
// Originali: 1 5 2 8 3 9 4 6; Pari (con copy_if): 2 8 4 6
```

std::back_inserter(pari):

dice: "aggiungi elementi alla fine del vector usando push_back()".

Senza back_inserter, il vector di destinazione deve avere già lo spazio allocato.

Applicare logica su un container: std::transform

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

int main() {
    std::vector<std::string> parole = {"ciao", "mondo", "test"};
    std::vector<int> lunghezze(3);

    // Calcola lunghezza di ogni parola
    std::transform(parole.begin(), parole.end(),
        lunghezze.begin(),
        [](const std::string& s) { return s.length(); });

    std::cout << "Parole: ";
    for (const auto& p : parole) std::cout << p << " ";
    std::cout << "\n";

    std::cout << "Lunghezze: ";
    for (const auto& len : lunghezze) std::cout << len << " ";
    std::cout << "\n";

    return 0;
}
// stampa: Lunghezze: 4 5 4
```


Riempire: std::fill

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> numeri(5);

    std::fill(numeri.begin(), numeri.end(), 10);

    std::cout << "Dopo fill(10): ";
    for (int x : numeri) std::cout << x << " ";
    std::cout << "\n";

    // Riempi solo parte
    std::fill(numeri.begin() + 1, numeri.begin() + 4, 20);

    std::cout << "Dopo fill(20) su parte: ";
    for (int x : numeri) std::cout << x << " ";
    std::cout << "\n";

    return 0;
}

// Dopo fill(10): 10 10 10 10 10
// Dopo fill(20) su parte: 10 20 20 20 10
```

Esercizio

- Dato un insieme di numeri `std::vector<int> numeri = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5};`
 - Estrarre i numeri pari,
 - Raddoppiare ogni numero estratto
 - Ordinate il risultato
 - Stampate il risultato
- Dato un insieme di nomi `std::vector<std::string> nomi = {"Alice", "Bob", "Charlie", "Diana", "Eve", "Frank"};`
 - Estrarre i nomi con lunghezza > 4
 - Convertire in maiuscolo con `std::to_upper`
 - Ordinare alfabeticamente
 - Invertire la lista ottenuta
 - Stampare il risultato

Utility

- `Std::optional`
- `Std::pair`
- `Std::tuple`

Optional

```
#include <iostream>
#include <optional>

int main() {
    std::optional<int> risultato = 42;

    if (risultato.has_value()) {
        std::cout << "Valore: " << risultato.value() << "\n";
    }

    std::optional<int> nulla; // Assente

    if (!nulla.has_value()) {
        std::cout << "Nessun valore\n";
    }

    return 0;
}

// valore: 42; Nessun valore
```

Quando usare `std::optional`

- Optional è usato quando dovete ritornare un valore, ma non è detto che ci sia.

Esempio di uso

```
#include <iostream>
#include <optional>
#include <string>

// Cerca elemento, ritorna optional
std::optional<int> trovaPosizione(const std::string& testo, char c) {
    for (int i = 0; i < testo.length(); i++) {
        if (testo[i] == c) {
            return i; // Trovato
        }
    }
    return std::nullopt; // Non trovato
}

int main() {
    auto pos1 = trovaPosizione("ciao", 'a');
    if (pos1) {
        std::cout << "'a' trovata in posizione: " << *pos1 << "\n";
    }

    auto pos2 = trovaPosizione("ciao", 'z');
    if (!pos2) {
        std::cout << "'z' non trovata\n";
    }

    return 0;
}

// 'a' trovata in posizione: 2; 'z' non trovata
```

Gestion errori avanzata da c++ 23: std::expected

```
#include <iostream>
#include <expected>
#include <string>

// Funzione che divide due numeri
// Ritorna: int (successo) oppure std::string (errore)
std::expected<int, std::string> dividi(int a, int b) {
    if (b == 0) {
        return std::unexpected("Errore: divisione per zero!");
    }
    return a / b;
}

int main() {
    std::cout << "Test 1: Divisione valida (10 / 2)\n";
    auto risultato1 = dividi(10, 2);

    if (risultato1) {
        std::cout << "Successo: " << risultato1.value() << "\n\n";
    } else {
        std::cout << "Errore: " << risultato1.error() << "\n\n";
    }

    std::cout << "Test 2: Divisione per zero (10 / 0)\n";
    auto risultato2 = dividi(10, 0);

    if (risultato2) {
        std::cout << "Successo: " << risultato2.value() << "\n\n";
    } else {
        std::cout << "Errore: " << risultato2.error() << "\n\n";
    }

    std::cout << "Test 3: Altro errore (20 / 0)\n";
    auto risultato3 = dividi(20, 0);

    if (!risultato3) {
        std::cout << "Errore riportato: " << risultato3.error() << "\n";
    }

    return 0;
}
```

Output:

Test 1: Divisione valida (10 / 2)

Successo: 5

Test 2: Divisione per zero (10 / 0)

Errore: Errore: divisione per zero!

Test 3: Altro errore (20 / 0)

Errore riportato: Errore: divisione per zero!

Coppie di valore: std::pair

• Cos'è?

- Una **coppia di due valori** (anche di tipi diversi). Ha `.first` e `.second`.
- Molto utile per ritornare più valori da una funzione

```
#include <iostream>
#include <utility>
```

```
int main() {
    // Coppia: stringa + numero
    std::pair<std::string, int> persona = {"Alice", 25};

    std::cout << "Nome: " << persona.first << "\n";
    std::cout << "Eta: " << persona.second << "\n";

    return 0;
}
// Nome: Alice; Eta: 25
```


Come si crea std::pair

// Modo 1: costruttore

```
std::pair<std::string, int> p1("Bob", 30);
```

// Modo 2: initializer list

```
std::pair<std::string, int> p2 = {"Charlie", 22};
```

// Modo 3: std::make_pair (deduce i tipi)

```
auto p3 = std::make_pair("Diana", 28);
```

Std::pair: binding (c++ 17)

// Con std::pair

auto [nemici, danno_tot] = statistiche(player); //  Structured binding

// Equivalente a

std::pair<**int**, **int**> result = statistiche(player);

int nemici = result.first;

int danno_tot = result.second;

Std::tuple

• Cos'è?

- Una collezione di valori di tipi potenzialmente diversi. È come `std::pair`, ma con più di due elementi.

```
#include <iostream>
```

```
#include <tuple>
```

```
int main() {
```

```
    // Tupla: nome, eta, città
```

```
    std::tuple<std::string, int, std::string> persona = {"Alice", 25, "Roma"};
```

```
    std::cout << "Nome: " << std::get<0>(persona) << "\n";
```

```
    std::cout << "Eta: " << std::get<1>(persona) << "\n";
```

```
    std::cout << "Citta: " << std::get<2>(persona) << "\n";
```

```
    return 0;
```

```
}
```

```
// Nome: Alice; Eta: 25; Citta: Roma
```

Std::tuple: come si crea

// Modo 1: costruttore

```
std::tuple<int, std::string, double> t1(42, "ciao", 3.14);
```

// Modo 2: initializer list

```
std::tuple<int, std::string, double> t2 = {42, "ciao", 3.14};
```

// Modo 3: std::make_tuple (deduce i tipi)

```
auto t3 = std::make_tuple(42, "ciao", 3.14);
```

Std::tuple: binding (c++17)

```
#include <iostream>
#include <tuple>

int main() {
    auto persona = std::make_tuple("Charlie", 22, "Napoli");

    // Scomponi la tupla nelle variabili
    auto [nome, eta, citta] = persona; // ✅ Structured binding

    std::cout << nome << " ha " << eta << " anni, vive a " << citta << "\n";

    return 0;
}
```

Output:

Charlie ha 22 anni, vive a Napoli

Esercizio

- Data una mappa `id_studente -> nome`,
 - scrivere una funzione che ritorna `optional<string>` (nome dello studente oppure nulla)
 - Testare con ID se esiste oppure no.
- Crea una funzione che ritorna `pair<int, int>`, es: `get_posizione`.
 - Ritornate una posizione fissa (es: 10, 20).
 - Stampate le coordinate. Usate il binding.
- Crea una tupla con: nome, livello, salute.
 - Usa binding per accedere ai dati
 - Stampa le informazioni

Esercizio opzionale

- Ritorna un `optional<pair<int, int>>` per coordinate.
 - Se le coordinate sono valide (entrambe > 0) ritorna il pair, altrimenti `nullopt`

Numeric

- Riduzioni
- Sequenze

Accumulatori: `std::accumulate`

- Usiamo `accumulate` quando vogliamo ridurre una collezione a un singolo valore oppure filtrare e trasformare in un unico passaggio.
- Esempi:
 - somma solo i numeri pari di una collezione, moltiplicati per 2
 - Concatena tutte le string di una lista.

Accumulatori: std::accumulate

```
#include <iostream>
#include <vector>
#include <string>
#include <numeric>

int main() {
    std::vector<std::string> parole = {"Ciao", " ", "mondo", "!"};

    // Concatena tutte le stringhe
    std::string risultato = std::accumulate(parole.begin(), parole.end(),
                                             std::string(""));
    std::cout << risultato << '\n'; // "Ciao mondo!"

    return 0;
}
```

Accumulate

```
#include <iostream>
#include <vector>
#include <numeric>

int main() {
    std::vector<int> numeri = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    // Somma solo i numeri pari, moltiplicati per 2
    int risultato = std::accumulate(numeri.begin(), numeri.end(), 0,
        [](int acc, int n) {
            if (n % 2 == 0) {
                return acc + (n * 2); // Filtra e trasforma
            }
            return acc;
        });

    std::cout << "Somma (pari * 2): " << risultato << "\n";
    // (2*2) + (4*2) + (6*2) + (8*2) + (10*2) = 60

    return 0;
}
```

Generatori di sequenze: std::iota

- Usato quando dobbiamo generare automaticamente un intervallo di valori fatto con certe regole
- Esempio:
 - Creare indici per iterare su collezioni
 - Inizializzare una collezione con dei numeri
 - Generare ID sequenziali (per mappe)

Generatori di sequenze: std::iota

```
#include <iostream>
#include <vector>
#include <numeric>

int main() {
    std::vector<std::string> nomi = {"Alice", "Bob", "Charlie", "Diana"};
    std::vector<int> indici(nomi.size());

    // Crea indici: 0, 1, 2, 3
    std::iota(indici.begin(), indici.end(), 0);

    // Usa gli indici per accedere
    for (int idx : indici) {
        std::cout << idx << ": " << nomi[idx] << '\n';
    }

    return 0;
}

// 0: Alice
// 1: Bob
// 2: Charlie
// 3: Diana
```

Esercizi

- Create una sequenza di numeri e sommate i valori.
- Create una sequenza di numeri e sommate solo i valori pari.
- Opzionale: hai un carrello della spesa con questi prodotti e prezzi:
 - {"Mela": 0.50, "Pane": 1.20, "Latte": 0.90}
 - Crea 2 vettori: per i nomi e per i prezzo
 - Usa accumulate per calcolare il totale
 - Calcola sconto 10%
 - Calcola prezzo finale
 - Stampa
 - Carrello:
 - - Mela: €0.50
 - - Pane: €1.20
 - - Latte: €0.90
 - ---
 - Totale: €2.60
 - Sconto (10%): €0.26
 - Prezzo finale: €2.34

Generatori di sequenze: std::iota

```
#include <iostream>
#include <vector>
#include <numeric>

struct Utente {
    int id;
    std::string nome;
};

int main() {
    std::vector<int> ids(5);
    std::iota(ids.begin(), ids.end(), 1000); // ID da 1000 in poi

    std::vector<Utente> utenti;
    std::vector<std::string> nomi = {"Alice", "Bob", "Charlie", "Diana", "Eve"};

    for (size_t i = 0; i < nomi.size(); ++i) {
        utenti.push_back({ids[i], nomi[i]});
    }

    for (const auto& u : utenti) {
        std::cout << "ID " << u.id << ": " << u.nome << '\n';
    }

    return 0;
}

// ID 1000: Alice
// ID 1001: Bob
// ID 1002: Charlie
// ID 1003: Diana
// ID 1004: Eve
```

Date e Time

- Duration
- Clock
- Time point
- Calendario

Durata

```
#include <iostream>
#include <chrono>

int main() {
    using namespace std::chrono;

    // Crea durate
    auto d1 = 5s;           // 5 secondi
    auto d2 = 1000ms;        // 1000 millisecondi
    auto d3 = 2h;            // 2 ore
    auto d4 = 30min;         // 30 minuti

    std::cout << "5 secondi = " << d1.count() << " sec\n";
    std::cout << "100 ms = " << d2.count() << " ms\n";
    std::cout << "2 ore = " << d3.count() << " h\n";

    // Converti tra unità
    auto d2_in_sec = duration_cast<seconds>(d2);
    std::cout << "1000ms in secondi: " << d2_in_sec.count() << "s\n";

    return 0;
}
```

- 5 secondi = 5 sec
- 1000 ms = 1000 ms
- 2 ore = 2 h
- 1000ms in secondi: 1s



Template programming

Essere generici risparmia codice

Cosa è un template

- Modelli per evitare di ripetere sempre la stessa logica per diversi tipi con verifica a compile time.

```
int max(int a, int b) { return a > b ? a : b; }  
double max(double a, double b) { return a > b ? a : b; }  
// ... ripetuto N volte
```

// Soluzione:

```
template<typename T>  
T max(T a, T b) { return a > b ? a : b; }
```

...ma che succede se passo tipi differenti?

```
Template<typename T>  
T max(T a, T b) { return a > b ? a : b; }
```

```
max(std::string("hello"), std::string("world")); // OK  
max(Point{1,2}, Point{3,4}); // ERRORE di 50 righe!
```

error: invalid operands to binary expression ('Point' and 'Point')

```
return a > b ? a : b;
```

~ ^ ~

note: in instantiation of function template specialization 'max<Point>' requested here

```
max(Point{1,2}, Point{3,4});
```

^

note: candidate template ignored: could not match 'type-parameter-0-0 *' against 'Point'

note: candidate template ignored: could not match 'basic_string<...>' against 'Point'

note: candidate function template not viable: no known conversion from 'Point' to 'const std::reverse_iterator<...>' for 1st argument

[... continua con decine di overload di operator> da STL ...]

Template con più parametri

```
template<typename T, typename U>
struct Pair {
    T first;
    U second;

    Pair(T f, U s) : first(f), second(s) {}
};

Pair<string, int> person{"Alice", 25};
Pair<int, double> coordinate{10, 3.14};
```

E se volessi un numero di parametri indefinito?

- Possiamo farlo con i variadic template:

```
template<typename... Args> // numero arbitrario di tipi!
```

```
void print(Args... args) {  
    (cout << ... << args); // fold expression  
}
```

```
print("Hello", " ", "World", " ", 42, " ", 3.14);
```

- ```
print("A", "B", "C");
```

```
// Si espande in:
```

```
(std::cout << "A") << "B") << "C")
```

```
// ^^^^^ primo step
```

```
// ^^^^^ secondo step
```

```
// ^^^^^ terzo step
```

# In viaggio verso universal reference

---

- Il problema:

```
template<typename T>
void add_to_vector(std::vector<T>& vec, T value) {
 vec.push_back(value);
}
```

```
std::vector<std::string> names;
std::string name = "Alice";
add_to_vector(names, name); // Funziona!
```

```
// -----
```

```
add_to_vector(names, name);
add_to_vector(names, std::string("Bob"));
add_to_vector(names, "Charlie");
```

```
// Copia name
// Copia temporaneo!
// Crea string, poi copia!
```

# In viaggio verso universal reference

---

- Mi scrivo un nuovo template (aumentano...). E non funziona sempre (const fallisce)

```
template<typename T>
void add_to_vector(std::vector<T>& vec, const T& value) {
 vec.push_back(value); // Sempre copia!
}
```

// Implementazione semplificata di push\_back

```
void push_back(const T& value) {
 // Devo creare un nuovo elemento nel vector
 // Ho solo una const reference a value
 // NON POSSO rubare le risorse da value (è const!)
 // Devo per forza COPIARE
 new (ptr) T(value); // copy constructor
}
```



# La soluzione: universal reference

---

- Lasciamo fare al compilatore il giusto passaggio di parametri evitando la copia dove possibile.

```
template<typename T>
void add_to_vector(std::vector<T>& vec, T&& value) {
 vec.push_back(std::forward<T>(value));
}
```

- Il reference collapsing:
- **La regola magica:**
  - Se passi un **lvalue**, T&& diventa T& (lvalue reference)
  - Se passi un **rvalue**, T&& diventa T&& (rvalue reference)

# Universal reference

---

```
template<typename T>
void add_to_vector(std::vector<T>& vec, T&& value) {
 vec.push_back(value); // ❌ value è sempre lvalue qui!
}
```

```
template<typename T>
void add_to_vector(std::vector<T>& vec, T&& value) {
 vec.push_back(std::forward<T>(value)); // ✅ Preserva!
}
```

std::forward<T> fa:

Se T è std::string& (era lvalue) → ritorna lvalue reference

Se T è std::string (era rvalue) → ritorna rvalue reference

# Universal reference

---

```
std::string name = "Alice";
```

```
// Caso 1: lvalue
```

```
add_to_vector(names, name);
```

```
// T = std::string&
```

```
// T&& = std::string& && → std::string&
```

```
// forward<std::string&>(value) → lvalue reference
```

```
// push_back fa COPIA ✓
```

```
// Caso 2: rvalue
```

```
add_to_vector(names, std::string("Bob"));
```

```
// T = std::string
```

```
// T&& = std::string&&
```

```
// forward<std::string>(value) → rvalue reference
```

```
// push_back fa MOVE ✓
```

# Concept

- E se potessi dire al compilatore che tipi accettare nel template ? Dal c++ 20 posso farlo con i concept

```
template<typename T>
concept Addable = requires(T a, T b) {
 a + b; // T deve supportare operator+
};
```

```
template<Addable T>
T add(T a, T b) {
 return a + b;
}
```

```
add(5, 3); //  OK
add(Point{}, Point{}); //  Errore CHIARO!
```

```
Errore con concepts.
```

```
error: cannot call function 'add'
 add(Point{}, Point{});
 ^~~
```

```
note: constraints not satisfied
note: concept 'Addable<Point>' was not satisfied
note: the required expression 'a + b' is invalid
```

# Anatomia del concept

---

- `template<typename T>`  
`concept Printable = requires(T obj) {`  
    `std::cout << obj; // deve esistere operator`  
`};`

Cosa succede:

Il compilatore non esegue il codice

Controlla solo se il codice compila

Se compila → `true`, altrimenti → `false`

# Esempio di concept

---

```
template<typename T>
concept Comparable = requires(T a, T b) {
 { a < b } -> std::convertible_to<bool>; // ritorna qualcosa convertibile a bool
 { a == b } -> std::convertible_to<bool>;
};
```

- Li posso combinare:

```
template<typename T>
concept Number = std::integral<T> || std::floating_point<T>;
```

```
template<typename T>
concept SignedNumber = Number<T> && std::is_signed_v<T>;
```

# Header <concept>

---

```
#include <concepts>

// Type categories
std::integral<int> // true
std::floating_point<double> // true
std::signed_integral<int> // true

// Comparisons
std::equality_comparable<T> // ha ==
std::totally_ordered<T> // ha <, >, <=, >=, ==, !=

// Object concepts
std::copyable<T> // può essere copiato
std::movable<T> // può essere mosso
std::destructible<T> // ha destructor pubblico

// Callable concepts
std::invocable<F, Args...> // F è chiamabile con Args
std::predicate<F, Args...> // F ritorna bool
```

# Esercizi

---

- Scrivere un template `Box<T>` che ha un `get` e un `set`. Usate universal reference per il costruttore e la `set`. Riflettete sul perché è necessario. Se non lo usate, che succede?
- Dato l'esercizio di prima aggiungete un concept che limita il `T` solo a tipi che sono stringhe
- Adesso modificate il concept per accettare anche tipi che sono convertibili a string.



# Ranges e programmazione funzionale

---

- **Stile Imperativo (Tradizionale)**

- *Dico al computer COME fare le cose, passo per passo*

```
std::vector<int> numbers = {1, 2, 3, 4, 5};
std::vector<int> result;

// COME: istruzioni step-by-step
for (int i = 0; i < numbers.size(); i++) {
 if (numbers[i] % 2 == 0) { // se pari
 int doubled = numbers[i] * 2; // moltiplica per 2
 result.push_back(doubled); // aggiungi al risultato
 }
}
```

- **Stile Funzionale (Dichiarativo)**

- *Dico al computer COSA voglio, non come ottenerlo*

```
auto result = numbers
 | filter(is_even) // COSA: tieni i pari
 | transform(double_it) // COSA: raddoppiali
 | to_vector(); // COSA: metti in un vector
```

# La compattezza dei ranges

- Gli iteratori spesso sono verbosi

```
#include <algorithm>
#include <vector>
#include <iostream>

int main() {
 std::vector<int> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

 // Voglio filtrare i pari, moltiplicarli per 2, e prendere i primi 3

 // Con algoritmi classici: orribile!
 std::vector<int> temp1;
 std::copy_if(numbers.begin(), numbers.end(),
 std::back_inserter(temp1),
 [](int x) { return x % 2 == 0; });

 std::vector<int> temp2;
 std::transform(temp1.begin(), temp1.end(),
 std::back_inserter(temp2),
 [](int x) { return x * 2; });

 std::vector<int> result;
 std::copy_n(temp2.begin(), 3, std::back_inserter(result));

 for (int x : result) {
 std::cout << x << " "; // Output: 4 8 12
 }
}
```

```
#include <ranges>
#include <vector>
#include <iostream>
```

```
int main() {
 std::vector<int> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

 // Stesso risultato, stile dichiarativo!
 auto result = numbers
 | std::views::filter([](int x) { return x % 2 == 0; })
 | std::views::transform([](int x) { return x * 2; })
 | std::views::take(3);

 for (int x : result) {
 std::cout << x << " "; // Output: 4 8 12
 }
}
```

Nota: non si usa più begin/end

# Cosa è un range

---

- Qualsiasi cosa si può iterare. Es: vector, string, array.
- I ranges funzionano con il concetto di vista (view):
  - Un range "lazy" (non fa nulla finché non iteriamo)
  - Non possiede i dati (punta al range originale)
  - Zero-cost: nessuna allocazione

```
std::vector<int> v = {1, 2, 3, 4, 5};
```

```
// Crea una view, non copia, né alloca!
```

```
auto evens = v | std::views::filter([](int x) { return x % 2 == 0; });
```

```
// Nulla è stato computato ancora!
```

```
// Solo ora itera e filtra
```

```
for (int x : evens) {
 std::cout << x; // 2 4
}
```

# Le view più usate

---

// 1. filtra

```
std::vector<int> v = {1, 2, 3, 4, 5, 6};
auto evens = v | std::views::filter([](int x) { return x % 2 == 0; });
// evens: 2, 4, 6
```

// 2. trasforma

```
std::vector<int> v = {1, 2, 3};
auto doubled = v | std::views::transform([](int x) { return x * 2; });
// doubled: 2, 4, 6
```

// 3. prendi i primi N elementi

```
std::vector<int> v = {1, 2, 3, 4, 5};
auto first3 = v | std::views::take(3);
// first3: 1, 2, 3
```

// 4. drop - salta i primi N elementi

```
std::vector<int> v = {1, 2, 3, 4, 5};
auto skip2 = v | std::views::drop(2);
// skip2: 3, 4, 5
```

# Le view più usate

---

// 5. inverti

```
std::vector<int> v = {1, 2, 3};
```

```
auto rev = v | std::views::reverse;
```

```
// rev: 3, 2, 1
```

// 6. Key-Value per le mappe

```
std::map<std::string, int> m = {"a", 1}, {"b", 2};
```

```
auto keys = m | std::views::keys; // "a", "b"
```

```
auto values = m | std::views::values; // 1, 2
```

// 7. generatore (iota)

```
auto numbers = std::views::iota(1, 6); // 1, 2, 3, 4, 5
```

```
auto infinite = std::views::iota(1); // 1, 2, 3, 4, ... ∞ - ricordate che è lazy!
```

# La potenza è data dalla composizione

```
• #include <ranges>
• #include <vector>
• #include <iostream>

• int main() {
• std::vector<int> v = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

• // Pipeline complessa, leggibile
• auto result = v
• | std::views::filter([](int x) { return x % 2 == 0; }) // pari
• | std::views::transform([](int x) { return x * x; }) // quadrato
• | std::views::take(3) // primi 3
• | std::views::reverse; // inverti

• for (int x : result) {
• std::cout << x << " "; // 64 16 4
• }

• // Risultato: 36 16 4
```

# Esercizio

---

• Hai una lista di studenti con nome e voto. Crea una pipeline che:

- Filtra solo gli studenti con voto  $\geq 18$  (promossi)
- Prende i primi 3
- Estrae solo i loro nomi
- Li stampa

```
#include <ranges>
#include <vector>
#include <string>
#include <iostream>
```

```
struct Student {
 std::string name;
 int grade;
};
```

# La gestione moderna degli errori

---

- Problema delle eccezioni: unwinding stack, puoi dimenticare il try-catch.

- Problema di `std::optional`

```
std::optional<int> parse_int(const std::string& s) {
 try {
 return std::stoi(s);
 }
 catch (...) {
 return std::nullopt; // Fallito, ma perché?
 }
}

auto result = parse_int("abc");
if (!result) {
 // È fallito, ma non so perchè
 // Formato errato? Overflow? Altro?
}
```

- Rafforziamo la gestione errori con `std::expected` (dal c++ 23)



# Std::expected

```
#include <expected>
#include <string>

enum class ParseError {
 EmptyString,
 InvalidFormat,
 Overflow
};

std::expected<int, ParseError> parse_int(const std::string& s) {
 if (s.empty()) {
 return std::unexpected(ParseError::EmptyString);
 }

 if (!std::all_of(s.begin(), s.end(), ::isdigit)) {
 return std::unexpected(ParseError::InvalidFormat);
 }

 try {
 return std::stoi(s);
 }
 catch (...) {
 return std::unexpected(ParseError::Overflow);
 }
}
```

```
auto result = parse_int("42");

if (result) {
 // Successo!
 std::cout << "Value: " << *result << "\n";
}
else {
 // Errore, so quale!
 switch (result.error()) {
 case ParseError::EmptyString:
 std::cerr << "String is empty\n";
 break;
 case ParseError::InvalidFormat:
 std::cerr << "Invalid number format\n";
 break;
 case ParseError::Overflow:
 std::cerr << "Number too large\n";
 break;
 }
}
```



# Multithreading e parallelismo

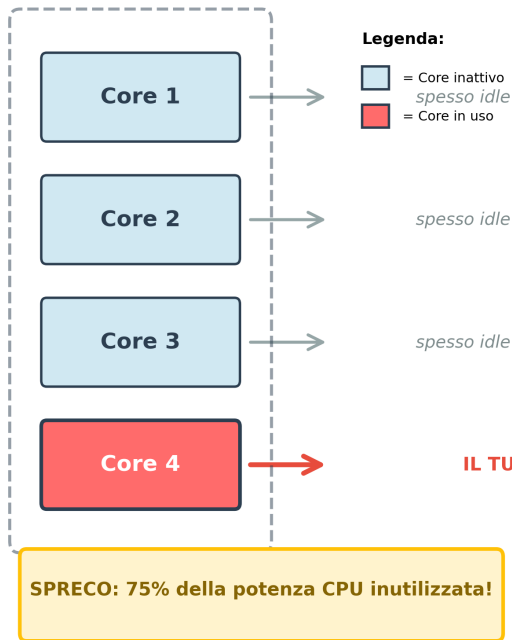
---

La vita complicata del programmatore moderno

# Perché usare i thread

## CPU Moderno con 4 Core

Programma Single-Thread



// Single-thread: lento! ✗

```
void process_images(const std::vector<Image>& images) {
 for (const auto& img : images) {
 apply_filter(img); // 100ms per immagine
 resize(img); // 50ms
 save(img); // 30ms
 }
}
```

// 100 immagini × 180ms = 18 secondi su 1 core

// Ma hai 4 cores disponibili! → potresti finire in ~4.5 secondi!

# I thread nel c++ 11

```
#include <iostream>
#include <thread>

void hello() {
 std::cout << "Hello from thread!\n";
}

int main() {
 std::thread t(hello); // Crea e avvia il thread

 std::cout << "Hello from main!\n";

 t.join(); // Aspetta che il thread finisca

 return 0;
}
```

Output(ordine non deterministico!) :

Hello from main!  
Hello from thread!

Oppure

Hello from thread!  
Hello from main!

Il sistema operativo ha lo scheduling e decide l'ordine in base a diversi fattori (es. interrupt, priorità, core liberi) l'ordine di esecuzione dei thread stessi.

Se non mettete join() -> crash al termine.

# Thread con parametri

---

```
#include <iostream>
#include <thread>

void print_sum(int a, int b) {
 std::cout << a << " + " << b << " = " << (a + b) << "\n";
}

int main() {
 std::thread t(print_sum, 5, 3); // Passa parametri
 t.join();

 return 0;
}
```

# Thread con lambda

---

```
#include <iostream>
#include <thread>
#include <vector>

int main() {
 std::vector<int> data = { 1, 2, 3, 4, 5 };

 std::thread t([&data]() {
 int sum = 0;
 for (int x : data) {
 sum += x;
 }
 std::cout << "Sum: " << sum << "\n";
 });

 t.join();

 return 0;
}
```

# Cosa può andare storto?

```
#include <thread>
#include <iostream>
#include <chrono>

void dangling_with_detach() {
 std::vector<int> data = { 1, 2, 3, 4, 5 };
 int multiplier = 10;

 std::jthread t([&] { // ⚠ Cattura TUTTO per riferimento
 std::this_thread::sleep_for(std::chrono::seconds(2));

 std::cout << "Thread woke up, accessing data...\n";
 for (int x : data) { // 💀 data è stato distrutto!
 std::cout << x * multiplier << "\n"; // 💀 multiplier pure!
 }
 });

 t.detach(); // ⚠ Thread va per la sua strada

 std::cout << "Function exiting, destroying data and multiplier\n";
 // data e multiplier distrutti qui! 💀
 // Ma thread ancora vivo per altri 2 secondi!
}

int main() {
 dangling_with_detach();

 std::cout << "Main waiting...\n";
 std::this_thread::sleep_for(std::chrono::seconds(3));

 std::cout << "Main done\n";
 return 0;
}
```

Mai passare per riferimento un parametro a un thread se non si è sicuri del life time del parametro stesso!

Detach = il thread non è più joinable

# Cosa può andare storto?

```
#include <thread>
#include <iostream>

void work() {
 std::cout << "Thread working...\n";
}

void dangerous_function() {
 std::thread t(work);

 // Fai qualcosa che può lanciare eccezione
 if (error_condition) {
 throw std::runtime_error("Error!"); // 💀 BOOM!
 }

 t.join(); // ➡ Mai raggiunto!
}

int main() {
 try {
 dangerous_function();
 }
 catch (...) {
 std::cout << "Caught exception\n";
 }
 // 💀 Programma non arriva qui! std::terminate() chiamato!
}
```

Cosa succede:

1. Crea thread t
2. Eccezione lanciata
3. Stack unwinding inizia
4. Distruttore di t chiamato
5. t è ancora joinable(non è stato fatto join / detach)
6. ~thread() vede che t è joinable → chiama std::terminate()
7. Crash!



# Cosa può andare storto?

```
#include <iostream>
#include <thread>

int counter = 0; // Variabile condivisa!

void increment() {
 for (int i = 0; i < 100000; ++i) {
 ++counter; // NON È ATOMICO! ⚠️
 }
}

int main() {
 std::thread t1(increment);
 std::thread t2(increment);

 t1.join();
 t2.join();

 std::cout << "Counter: " << counter << "\n";
 // Ti aspetti 200000, ma ottieni... 137829? 156432? Random!
}
```

Non stiamo proteggendo la variabile condivisa tra i thread.

# Usate Jthread (c++20)

```
#include <thread>
#include <iostream>

void work() {
 std::cout << "Thread working...\n";
}

void safe_function() {
 std::jthread t(work); // ✅ jthread!

 if (error_condition) {
 throw std::runtime_error("Error!"); // ✅ OK!
 }

 // Non serve join esplicito
}

int main() {
 try {
 safe_function();
 }
 catch (...) {
 std::cout << "Caught exception\n"; // ✅ Raggiunto!
 }

 std::cout << "Program continues\n"; // ✅ Raggiunto!
}
```

## Cosa succede:

1. Crea jthread t
2. Eccezione lanciata
3. Stack unwinding inizia
4. Distruttore di t chiamato
5. ~jthread() fa join() (RAII)
6. Thread finisce
7. Eccezione propagata normalmente
8. Programma continua

# Mutex

---

```
#include <iostream>
#include <thread>
#include <mutex>

int counter = 0;
std::mutex mtx; // Protezione!

void increment() {
 for (int i = 0; i < 100000; ++i) {
 mtx.lock(); // Acquisisci il lock
 ++counter; // Sezione critica (protetta)
 mtx.unlock(); // Rilascia il lock
 }
}

int main() {
 std::thread t1(increment);
 std::thread t2(increment);

 t1.join();
 t2.join();

 std::cout << "Counter: " << counter << "\n"; // 200000 ✓
}
```

// Problema! Qualcosa va storto, tra lock e unlock (lancio eccezione) -> unlock non viene più chiamato con brutte conseguenze

# RAII in nostro aiuto

```
#include <iostream>
#include <thread>
#include <mutex>

int counter = 0;
std::mutex mtx;

void increment() {
 for (int i = 0; i < 100000; ++i) {
 std::lock_guard<std::mutex> lock(mtx); // RAII!
 ++counter;
 // unlock automatico quando lock_guard esce dallo scope
 }
}

int main() {
 std::thread t1(increment);
 std::thread t2(increment);

 t1.join();
 t2.join();

 std::cout << "Counter: " << counter << "\n"; // 200000 ✓
}
```

# Tipo di lock

---

- Std::lock\_guard
  - Non potete fare lock/unlock

```
void push(int value) {
 std::lock_guard<std::mutex> lock(mtx_);
 data_.push_back(value);
}
```
- Std::unique\_lock

```
void flexible() {
 std::unique_lock<std::mutex> lock(mtx); // Lock acquisito
 // Fai qualcosa
 lock.unlock(); // ✓ Unlock manuale possibile!

 // Fai qualcosa SENZA lock (sezione non critica)
 expensive_computation();

 lock.lock(); // ✓ Re-lock possibile!
 // Sezione critica di nuovo
} // unlock automatico alla fine (se ancora lockato)
```

# Tipo di lock

---

- Std::scoped\_lock (consiglio: usatelo sempre)
  - Per acquisire lock multipli

```
#include <mutex>

std::mutex mtx1, mtx2;

void thread1() {
 std::scoped_lock lock(mtx1, mtx2); // Acquisisce ENTRAMBI atomicamente!

 // Lavoro con entrambi i mutex
} // Unlock automatico di entrambi

void thread2() {
 std::scoped_lock lock(mtx1, mtx2); // Stesso ordine o diverso, funziona!

 // Lavoro
}

// NO DEADLOCK! ✓
```

# Il passaggio per riferimenti (lifetime)

---

```
void process_data(std::shared_ptr<std::vector<int>> data) {
 for (int x : *data) {
 std::cout << x << " ";
 }
 std::cout << "\n";
}

int main() {
 auto data = std::make_shared<std::vector<int>>({ 1, 2, 3, 4, 5 });

 std::jthread t(process_data, data); // ✅ shared_ptr copiato

 // data può andare out of scope, thread ha ancora accesso!
}
```

# Il passaggio per riferimenti (lifetime)

---

```
void process_data(std::unique_ptr<std::vector<int>> data) {
 // Thread ha ownership esclusivo
 for (int x : *data) {
 std::cout << x << " ";
 }
}

int main() {
 auto data = std::make_unique<std::vector<int>>>({ 1, 2, 3, 4, 5 });

 std::jthread t(process_data, std::move(data));

 // data è ora nullptr, ownership trasferito al thread
}
```



# Problema: non tenere a lungo il lock!

- Usate il lock solo per il minimo indispensabile. Altrimenti si finisce nel cosiddetto «lock contention»

```
#include <mutex>
#include <vector>
#include <fstream>
#include <iostream>

class Logger {
 std::vector<std::string> messages_;
 std::mutex mtx_;

public:
 void log_and_save(const std::string& msg) {
 std::lock_guard lock(mtx_); // Lock acquisito

 messages_.push_back(msg); // ✅ Veloce (microseconds)

 // ❌ I/O dentro il lock! (milliseconds/seconds)
 std::ofstream file("log.txt", std::ios::app);
 file << msg << "\n";
 file.close();

 // Lock rilasciato solo QUI!
 // Altri thread bloccati per TUTTO questo tempo!
 }
};
```

# Lock minimo

---

```
class Logger {
 std::vector<std::string> messages_;
 std::mutex mtx_;

public:
 void log_and_save(const std::string& msg) {
 {
 std::lock_guard lock(mtx_);
 messages_.push_back(msg); // ✅ Solo questo protetto
 } // Lock rilasciato QUI!

 // ✅ I/O fuori dal lock
 std::ofstream file("log.txt", std::ios::app);
 file << msg << "\n";
 file.close();

 // Altri thread possono procedere mentre facciamo I/O!
 }
};
```

# Esercizi

---

```
#include <thread>
#include <iostream>
#include <vector>
#include <mutex>

std::vector<int> shared_list;
std::mutex list_mtx;

// TODO: Implementa questa funzione
// Deve aggiungere i numeri da start a end nella lista condivisa
// Usa scoped_lock per proteggere l'accesso!
void add_numbers(int start, int end) {
 ...
}

int main() {
 // crea più thread che chiamano add_numbers

 // stampa la lista finale
 return 0;
}
```

# Esercizi

---

```
#include <thread>
#include <iostream>
#include <vector>
#include <string>
#include <chrono>
#include <mutex>

std::mutex cout_mtx; // Mutex per std::cout

// Simula processing di un file (sleep per 100ms)
void process_file(const std::string& filename) {
 // TODO:
 // 1. Stampa "Processing <filename>..." (protetto!)
 // 2. Simula lavoro: sleep_for 100ms
 // 3. Stampa "Finished <filename>" (protetto!)
}

int main() {
 std::vector<std::string> files = {
 "document1.txt", "document2.txt", "document3.txt",
 "document4.txt", "document5.txt"
 };

 auto start = std::chrono::high_resolution_clock::now();

 std::vector<std::jthread> threads;

 // TODO: Crea un thread per ogni file

 // join automatico

 auto end = std::chrono::high_resolution_clock::now();
 auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

 std::cout << "Tempo totale: " << duration.count() << "ms\n";
 std::cout << "(Sequenziale sarebbe stato ~500ms)\n";

 return 0;
}
```

# Parallelismo

---

- Il parallelismo è la capacità di dividere un task complesso in tanti compiti eseguiti parallelamente.
- Ad esempio: devo applicare un filtro su un'immagine grande. Posso dividere l'immagine in più blocchi ed eseguire il filtro parallelamente su ogni blocco grazie all'architettura multi core dei processori moderni.
- CONCURRENCY (Multithreading):
  - Thread 1: [Read DB] → [Lock] → [Write] → [Unlock]
  - Thread 2: [Wait] → [Lock] → [Write] → [Unlock]
  - Thread 3: [Wait] → [Lock] → [Write]
  - Thread si aspettano a vicenda (coordinazione)
- PARALLELISM (Data Parallelism):
  - Core 1: [Process items 0-250k]
  - Core 2: [Process items 250k-500k]
  - Core 3: [Process items 500k-750k]
  - Core 4: [Process items 750k-1M]

# Std::async

```
#include <future>
#include <iostream>

int compute() {
 // Simulazione calcolo pesante
 int sum = 0;
 for (int i = 0; i < 100'000'000; ++i) {
 sum += i;
 }
 return sum;
}

int main() {
 // Avvia in parallelo
 std::future<int> future = std::async(std::launch::async, compute);

 std::cout << "Main: doing other work while compute() runs...\n";

 // Ottieni il risultato (blocca se non ancora pronto)
 int result = future.get();

 std::cout << "Result: " << result << "\n";

 return 0;
}
```

async avvia la funzione in un thread separato.

Ritorna un future<T> (promessa di risultato futuro).

future.get() aspetta e ritorna il risultato.

Join automatico quando future viene distrutto.

# Parallel STL

---

- Dal c++17 sono state introdotte le policy di parallelismo in molti algoritmi di STL.

```
#include <algorithm>
#include <execution>
#include <vector>

std::vector<int> data(1'000'000);

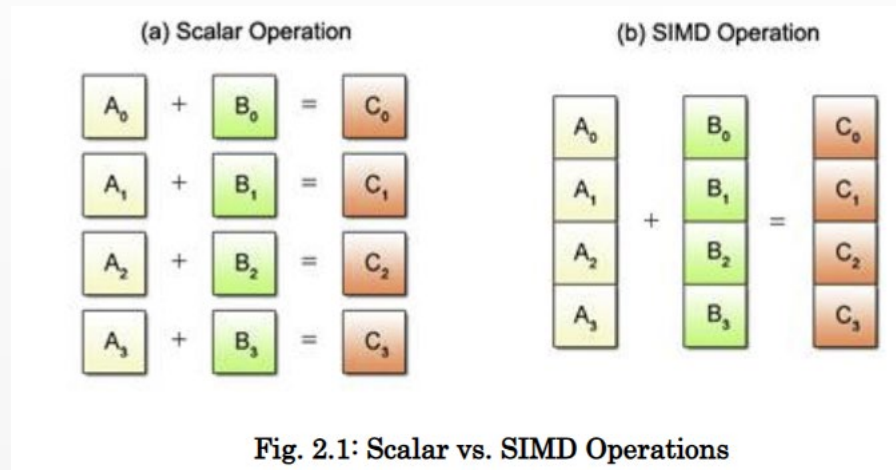
// Sequenziale (default, come sempre)
std::for_each(std::execution::seq,
 data.begin(), data.end(),
 [](int& x) { x *= 2; });

// Parallelo (multi-thread)
std::for_each(std::execution::par,
 data.begin(), data.end(),
 [](int& x) { x *= 2; });

// Parallelo + Vectorizzato (SIMD)
std::for_each(std::execution::par_unseq,
 data.begin(), data.end(),
 [](int& x) { x *= 2; });

// Solo vectorizzato (single thread + SIMD)
std::for_each(std::execution::unseq, // C++20
 data.begin(), data.end(),
 [](int& x) { x *= 2; });
```

# SIMD: Single Instruction, Multiple Data





# La policy di execution

---

- `Std::execution::par` -> ogni core prende un pezzo da elaborare
- `Std::execution::unseq (SIMD)` -> simd su un solo core:

```
std::for_each(std::execution::unseq,
 data.begin(), data.end(),
 [](int& x) { x *= 2; });
```

Un solo core, ma processa 8 elementi per volta con AVX2:

Core 1: `[[0-7]]` → `[[8-15]]` → `[[16-23]]` → ... → `[[999,992-999,999]]`

↑ 8 elementi processati con UNA istruzione SIMD

Tempo: ~1/8 del tempo sequenziale scalare

# La policy di execution

---

- Std::execution::par\_unseq -> SIMD su tutti i core
  - Come funziona (4 core + AVX2):
    - Ogni core usa SIMD per processare la sua fetta:
      - Core 1:  $[[0-7]] \rightarrow [[8-15]] \rightarrow \dots \rightarrow [[249,992-249,999]]$ 
        - $\uparrow$  SIMD
      - Core 2:  $[[250,000-250,007]] \rightarrow [[250,008-250,015]] \rightarrow \dots$ 
        - $\uparrow$  SIMD
      - Core 3:  $[[500,000-500,007]] \rightarrow \dots$ 
        - $\uparrow$  SIMD
      - Core 4:  $[[750,000-750,007]] \rightarrow \dots$ 
        - $\uparrow$  SIMD
    - Speedup teorico:  $4 \text{ core} \times 8 \text{ SIMD} = \sim 32x!$

# Esempio

---

//  Processing immagini (pixel indipendenti)

```
std::for_each(std::execution::par, pixels.begin(),
pixels.end(),
 [](auto& p) { apply_filter(p); });
```

//  Calcoli numerici (elementi indipendenti)

```
std::transform(std::execution::par, data.begin(), data.end(),
result.begin(),
 [](double x) { return expensive_math(x); });
```

//  Sort grande array

```
std::sort(std::execution::par, huge_array.begin(),
huge_array.end());
```

# Riepilogo dei casi

---

- Ho vari task che condividono dei dati e devono comunicare tra loro -> threads
  - Es: un server in attesa di una connessione, un produttore-consumatore
- Pochi task indipendenti e pesanti -> std::async
  - Es: all'avvio di un gioco carico texture, modelli, audio in parallelo
- Tanti dati, stesso lavoro su ognuno -> stl parallel
  - Es: 1 mil di pixel da processare
    - Policy ::par => default, usa tutti i core (es: sort, tranform)
    - Policy ::par\_unseq => operazioni matematiche pure, no alloc.

# Le domande da porsi

---

## Riepilogo in 3 Domande

**Domanda 1:** I task devono **condividere** dati o **coordinarsi**?

- **SÌ** → `jthread + mutex`
- **NO** → Vai a domanda 2

**Domanda 2:** Quanti task?

- **Pochi (2-10), pesanti, con risultato** → `std::async`
- **Tanti (migliaia+), stessa operazione** → Vai a domanda 3

**Domanda 3:** Che tipo di operazione?

- **Complessa** (string, if/else, allocazioni) → `std::execution::par`
- **Math puro** (no branch, no alloc) → `std::execution::par_unseq`

# Esercizi

```
#include <algorithm>
#include <execution>
#include <vector>
#include <iostream>
#include <chrono>

int main() {
 std::vector<int> data(10'000'000, 1);
 std::vector<int> result(data.size());

 // TODO: 1. Usa std::transform con execution::seq
 // Trasforma ogni elemento: x = x * 2 + 1
 auto start_seq = std::chrono::high_resolution_clock::now();

 // ... tuo codice qui

 auto end_seq = std::chrono::high_resolution_clock::now();

 // Reset result
 std::fill(result.begin(), result.end(), 0);

 // TODO: 2. Usa std::transform con execution::par
 // Stessa operazione
 auto start_par = std::chrono::high_resolution_clock::now();

 // ... tuo codice qui

 auto end_par = std::chrono::high_resolution_clock::now();

 // Reset result
 std::fill(result.begin(), result.end(), 0);

 // TODO: 3. Usa std::transform con execution::par_unseq
 // Stessa operazione (multi-thread + SIMD!)
 auto start_par_unseq = std::chrono::high_resolution_clock::now();

 // ... tuo codice qui

 auto end_par_unseq = std::chrono::high_resolution_clock::now();
```

```
// Calcola tempi
 auto seq_ms = std::chrono::duration_cast<std::chrono::milliseconds>(end_seq -
start_seq);
 auto par_ms = std::chrono::duration_cast<std::chrono::milliseconds>(end_par -
start_par);
 auto par_unseq_ms =
std::chrono::duration_cast<std::chrono::milliseconds>(end_par_unseq -
start_par_unseq);

 std::cout << "Sequential: " << seq_ms.count() << "ms (baseline)\n";
 std::cout << "Parallel: " << par_ms.count() << "ms (speedup: "
 << (float)seq_ms.count() / par_ms.count() << "x)\n";
 std::cout << "Parallel+SIMD: " << par_unseq_ms.count() << "ms (speedup: "
 << (float)seq_ms.count() / par_unseq_ms.count() << "x)\n";

 return 0;
}
```

Obiettivo: Confrontare seq, par e par\_unseq

# Esercizi

```
#include <algorithm>
#include <execution>
#include <vector>
#include <iostream>
#include <cstdlib>

struct Pixel {
 uint8_t r, g, b;
};

int main() {
 // Immagine 4K: 3840 × 2160 pixels
 std::vector<Pixel> image(3840 * 2160);

 // Inizializza con pixel casuali
 for (auto& p : image) {
 p.r = rand() % 256;
 p.g = rand() % 256;
 p.b = rand() % 256;
 }

 // TODO: Converti a grayscale in parallelo
 // Formula: gray = 0.299*R + 0.587*G + 0.114*B
 // Poi imposta R = G = B = gray

 // ... tuo codice qui

 // Verifica: primo pixel dovrebbe avere R = G = B
 std::cout << "Primo pixel dopo conversione:\n";
 std::cout << "R=" << (int)image[0].r
 << " G=" << (int)image[0].g
 << " B=" << (int)image[0].b << "\n";

 bool is_grayscale = (image[0].r == image[0].g && image[0].g == image[0].b);
 std::cout << "Grayscale: " << (is_grayscale ? "SI" : "NO") << "\n";

 return 0;
}
```

**Obiettivo:** Convertire immagine a colori in bianco e nero



# Design pattern

---

La buona architettura del software



# Cosa sono

---

- Un design pattern è una soluzione generale e riutilizzabile a un problema ricorrente nella progettazione del software. Non è codice pronto da copiare, ma uno schema — una descrizione di come organizzare classi, oggetti e le loro relazioni per risolvere un certo tipo di problema in modo pulito e manutenibile.
- L'idea è simile a quella di un architetto: quando deve progettare una scala, non la reinventa ogni volta da zero. Conosce degli schemi collaudati (a chiocciola, a rampa, a gradini sfalsati) e sceglie quello più adatto al contesto. Un design pattern è esattamente questo, applicato al codice.

# L'origine

---

- Nel 1994, quattro autori — Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides — pubblicarono il libro "*Design Patterns: Elements of Reusable Object-Oriented Software*". Il libro catalogava 23 pattern, ciascuno documentato con lo stesso formato: nome, intento, motivazione, struttura, partecipanti, conseguenze, implementazione, esempi.
- I quattro autori divennero noti come la **Gang of Four** (GoF), e il loro libro è ancora oggi il riferimento fondamentale sull'argomento. Anche se gli esempi originali erano in C++ e Smalltalk, i pattern sono indipendenti dal linguaggio: si applicano in Java, C#, Python, Rust e in qualsiasi contesto orientato a oggetti.
- È importante sapere che i 23 pattern GoF non sono gli unici esistenti. Nel tempo ne sono stati descritti molti altri (e alcuni dei 23 originali sono caduti in disuso). Ma i pattern GoF restano il vocabolario comune che ogni programmatore dovrebbe conoscere.

# Categorie di pattern

---

- I 23 pattern GoF sono divisi in tre famiglie, in base al tipo di problema che risolvono:
  - **Creazionali — *come creare gli oggetti***
    - Si occupano dei meccanismi di creazione, cercando di rendere il sistema indipendente da come i suoi oggetti vengono istanziati e composti.
    - Pattern GoF: Abstract Factory, Builder, Factory Method, Prototype, Singleton.
  - **Strutturali — *come comporre gli oggetti***
    - Si occupano di come classi e oggetti vengono combinati per formare strutture più grandi, mantenendo flessibilità e efficienza.
    - Pattern GoF: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy.
  - **Comportamentali — *come gli oggetti comunicano***
    - Si occupano degli algoritmi e della distribuzione delle responsabilità tra gli oggetti, definendo come interagiscono e come il flusso di controllo si muove tra di loro.
    - Pattern GoF: Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor.

# Factory

---

- **Problema:** il codice che *usa* un oggetto non dovrebbe sapere *quale tipo concreto* creare. Se ogni punto del programma decide da solo quale classe istanziare, un cambio o un'aggiunta costringe a modificare codice ovunque.
- **Soluzione:** si delega la creazione a una funzione (o classe) dedicata. Chi chiama riceve un oggetto attraverso un'interfaccia comune, senza conoscere il tipo concreto restituito. La decisione su *cosa* costruire è centralizzata in un unico punto.
- **Quando serve:** ogni volta che esistono più varianti di uno stesso concetto e la scelta fra esse dipende da un dato esterno (un file di configurazione, un input utente, uno stato di gioco).

# Factory

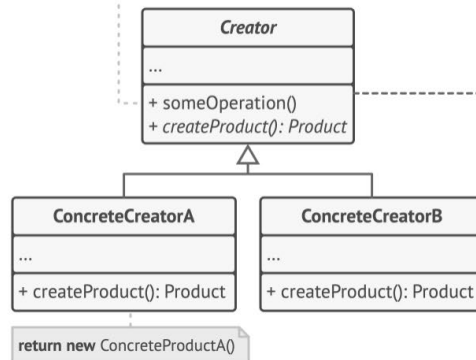
## Structure

3 The **Creator** class declares the factory method that returns new product objects. It's important that the return type of this method matches the product interface.

You can declare the factory method as `abstract` to force all subclasses to implement their own versions of the method. As an alternative, the base factory method can return some default product type.

Note, despite its name, product creation is **not** the primary responsibility of the creator. Usually, the creator class already has some core business logic related to products. The factory method helps to decouple this logic from the concrete product classes. Here is an analogy: a large software development company can have a training department for programmers. However, the primary function of the company as a whole is still writing code, not producing programmers.

```
Product p = createProduct()
p.doStuff()
```



4 **Concrete Creators** override the base factory method so it returns a different type of product.

Note that the factory method doesn't have to **create** new instances all the time. It can also return existing objects from a cache, an object pool, or another source.

1 The **Product** declares the interface, which is common to all objects that can be produced by the creator and its subclasses.

2 **Concrete Products** are different implementations of the product interface.

# Factory

```
#include <memory>
#include <string>
#include <print>

// --- Interfaccia comune ---

class Enemy {
public:
 virtual ~Enemy() = default;
 virtual auto name() const -> std::string = 0;
 virtual auto attack() const -> int = 0;
 virtual auto health() const -> int = 0;
};

// --- Tipi concreti ---

class Goblin final : public Enemy {
public:
 auto name() const -> std::string override { return "Goblin"; }
 auto attack() const -> int override { return 5; }
 auto health() const -> int override { return 30; }
};

class Skeleton final : public Enemy {
public:
 auto name() const -> std::string override { return "Skeleton"; }
 auto attack() const -> int override { return 8; }
 auto health() const -> int override { return 20; }
};

class Dragon final : public Enemy {
public:
 auto name() const -> std::string override { return "Dragon"; }
 auto attack() const -> int override { return 25; }
 auto health() const -> int override { return 200; }
};

// --- Factory: l'unico punto che conosce i tipi concreti ---

enum class EnemyType { goblin, skeleton, dragon };

auto make_enemy(EnemyType type) -> std::unique_ptr<Enemy> {
 switch (type) {
 case EnemyType::goblin: return std::make_unique<Goblin>();
 case EnemyType::skeleton: return std::make_unique<Skeleton>();
 case EnemyType::dragon: return std::make_unique<Dragon>();
 }
 std::unreachable(); // C++23: il compilatore sa che non arriviamo qui
}
```

```
int main() {
 auto wave = { EnemyType::goblin, EnemyType::goblin,
 EnemyType::dragon };

 for (auto type : wave) {
 auto enemy = make_enemy(type);
 std::println("Spawn: {} (ATK: {} HP: {})",
 enemy->name(), enemy->attack(), enemy->health());
 }
}
```

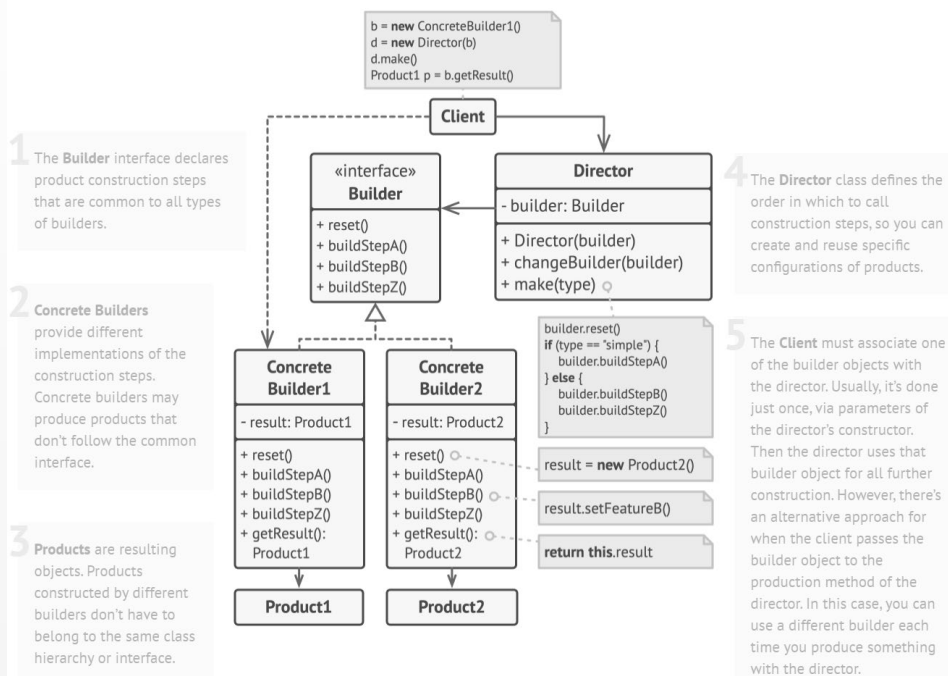
# Builder

---

- **Problema:** un oggetto complesso richiede molti parametri per essere costruito. Un costruttore con tanti argomenti è difficile da leggere, fragile da mantenere e non rende esplicito quali parametri sono obbligatori e quali opzionali.
- **Soluzione:** si costruisce l'oggetto passo dopo passo, attraverso una sequenza di chiamate leggibili, ognuna delle quali configura un singolo aspetto. Al termine, un'operazione finale assembla e restituisce l'oggetto completo. Ogni passo è opzionale o ha un valore predefinito ragionevole.
- **Quando serve:** quando la costruzione ha molte combinazioni possibili e si vuole che il codice di creazione sia leggibile come una descrizione a parole dell'oggetto.

# Builder

## Structure





# Builder

```
#include <string>
#include <vector>
#include <print>
#include <optional>

struct Vec3 {
 float x{}, y{}, z{};
};

enum class WeaponType { sword, bow, staff, claws };
enum class ArmorType { none, light, heavy };

// --- L'oggetto che vogliamo costruire ---

struct GameEnemy {
 std::string name;
 int health{ 100 };
 WeaponType weapon{ WeaponType::claws };
 ArmorType armor{ ArmorType::none };
 Vec3 spawn_point{};
 float speed{ 1.0f };
 std::vector<std::string> abilities;
 std::optional<std::string> loot;
};

// --- Builder con fluent interface ---

class EnemyBuilder {
public:
 explicit EnemyBuilder(std::string name) {
 enemy_.name = std::move(name);
 }

 auto with_health(int hp) -> EnemyBuilder& { enemy_.health = hp; return *this; }
 auto with_weapon(WeaponType w) -> EnemyBuilder& { enemy_.weapon = w; return *this; }
 auto with_armor(ArmorType a) -> EnemyBuilder& { enemy_.armor = a; return *this; }
 auto with_spawn_point(Vec3 pos) -> EnemyBuilder& { enemy_.spawn_point = pos; return *this; }
 auto with_speed(float s) -> EnemyBuilder& { enemy_.speed = s; return *this; }
 auto with_ability(std::string ability) -> EnemyBuilder& { enemy_.abilities.push_back(std::move(ability)); return *this; }
 auto with_loot(std::string item) -> EnemyBuilder& { enemy_.loot = std::move(item); return *this; }

 // build() trasferisce l'oggetto al chiamante
 auto build() -> GameEnemy {
 return std::move(enemy_);
 }

private:
 GameEnemy enemy_;
};
```

```
int main() {
 // Leggibile come una descrizione a parole
 auto boss = EnemyBuilder("Dragon Lord")
 .with_health(500)
 .with_weapon(WeaponType::claws)
 .with_armor(ArmorType::heavy)
 .with_speed(0.6f)
 .with_ability("fire_breath")
 .with_ability("tail_sweep")
 .with_loot("legendary_sword")
 .with_spawn_point({ 100.0f, 0.0f, 200.0f })
 .build();

 std::println("Boss: {} (HP:{} speed:{:.1f})", boss.name, boss.health,
 boss.speed);
 std::println("Abilities: {}", boss.abilities.size());

 // Nemico semplice: solo il nome e i default bastano
 auto minion = EnemyBuilder("Slime")
 .with_health(10)
 .build();

 std::println("Minion: {} (HP:{})", minion.name, minion.health);
}
```

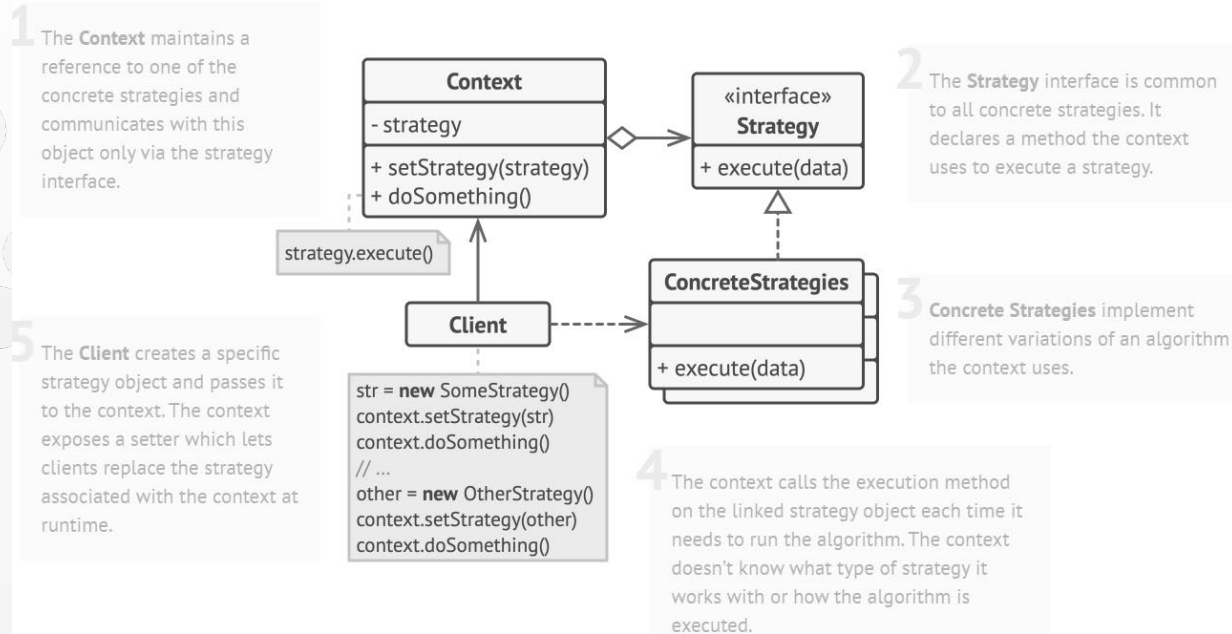
# Strategy

---

- **Problema:** un oggetto deve poter cambiare comportamento a runtime, ma codificare tutte le varianti con catene di if/else o switch rende il codice rigido e difficile da estendere.
- **Soluzione:** si estrae il comportamento variabile in un'entità separata (una funzione, un oggetto) che può essere sostituita dall'esterno. L'oggetto non sa *come* il comportamento è implementato, sa solo che può invocarlo.
- **Quando serve:** ogni volta che uno stesso soggetto deve comportarsi in modi diversi a seconda del contesto, e quei modi possono cambiare durante l'esecuzione del programma.

# Strategy

## Structure



# Strategy

```
#include <functional>
#include <string>
#include <print>
#include <cmath>

struct Vec3 {
 float x{}, y{}, z{};
};

struct World {
 Vec3 player_position{ 10.0f, 0.0f, 5.0f };
};

// --- Il nemico con uno Strategy sostituibile ---

class Enemy {
public:
 // Lo strategy è una funzione: nessuna gerarchia di classi necessaria
 using Strategy = std::function<void(Enemy&, const World&)>;

 std::string name;
 Vec3 position{};
 int health{ 100 };

 explicit Enemy(std::string name, Strategy ai)
 : name{ std::move(name) }, ai_{ std::move(ai) } {}

 void set_ai(Strategy new_ai) { ai_ = std::move(new_ai); }

 void update(const World& world) {
 if (ai_) ai_(*this, world);
 }

private:
 Strategy ai_;
};
```

# Strategy

---

```
// Pattuglia avanti e indietro
inline auto patrol(Vec3 a, Vec3 b) {
 return [a, b, going_to_b = true](Enemy& e, const World&) mutable {
 auto& target = going_to_b ? b : a;
 e.position.x += (target.x - e.position.x) * 0.1f;
 e.position.z += (target.z - e.position.z) * 0.1f;

 // Arrivato? Inverte direzione
 auto dx = target.x - e.position.x;
 auto dz = target.z - e.position.z;
 if (std::sqrt(dx * dx + dz * dz) < 0.5f) {
 going_to_b = !going_to_b;
 }
 std::println(" [patrol] {} at {:.1f}, {:.1f}", e.name, e.position.x, e.position.z);
 };
}
```

# Strategy

---

```
// Fugge dal giocatore
inline auto flee() {
 return [](Enemy& e, const World& w) {
 auto dx = e.position.x - w.player_position.x;
 auto dz = e.position.z - w.player_position.z;
 auto dist = std::sqrt(dx * dx + dz * dz);
 if (dist > 0.01f) {
 e.position.x += (dx / dist) * 0.8f;
 e.position.z += (dz / dist) * 0.8f;
 }
 std::println(" [flee] {} running! ({:.1f}, {:.1f})", e.name, e.position.x,
 e.position.z);
 };
}
```

# Strategy

---

```
// --- Uso ---

int main() {
 World world;

 auto goblin = Enemy("Goblin", patrol({ 0, 0, 0 }, { 5, 0, 5 }));

 std::println("--- Pattugliamento ---");
 for (int i = 0; i < 3; ++i) goblin.update(world);

 std::println("--- Il player si avvicina: inseguimento! ---");
 goblin.set_ai(chase());
 for (int i = 0; i < 3; ++i) goblin.update(world);

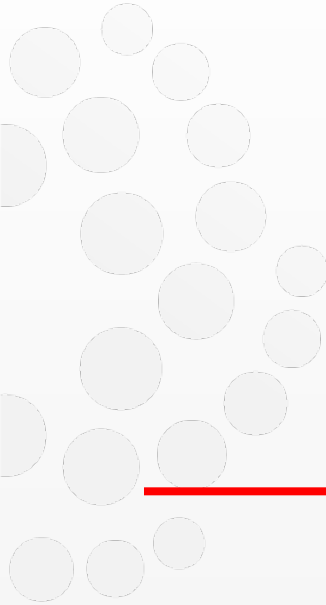
 std::println("--- Poca vita: fuga! ---");
 goblin.health = 5;
 goblin.set_ai(flee());
 for (int i = 0; i < 3; ++i) goblin.update(world);
}
```

# Strategy

---

```
// Insegue il giocatore
inline auto chase() {
 return [](Enemy& e, const World& w) {
 auto dx = w.player_position.x - e.position.x;
 auto dz = w.player_position.z - e.position.z;
 auto dist = std::sqrt(dx * dx + dz * dz);
 if (dist > 0.1f) {
 e.position.x += (dx / dist) * 0.5f;
 e.position.z += (dz / dist) * 0.5f;
 }
 std::println(" [chase] {} -> player {:.1f}, {:.1f})", e.name, e.position.x,
 e.position.z);
 };
}
```





# Grazie

---

Cristiano.Chieppa@smartsecuritysw.it